

第 13 章 系统设计

系统设计是系统分析的延伸与拓展。系统分析阶段解决“做什么”的问题，而系统设计阶段解决“怎么做”的问题。同时，它也是系统实施的基础，为系统实施工作做好铺垫。合理的系统设计方案既可以保证系统的质量，也可以提高开发效率，确保系统实施工作的顺利进行。

系统设计阶段又称为物理设计阶段，它是信息系统开发过程中一个非常重要的阶段。其任务是根据系统规格说明书中规定的功能要求，考虑实际条件，具体设计实现逻辑模型的技术方案，也就是设计新系统的物理模型，为下一阶段的系统实施工作奠定基础。

13.1 系统设计概述

系统设计的目标是根据系统分析的结果，完成系统的构建过程。其主要目的是绘制系统的蓝图，权衡和比较各种技术和实施方法的利弊，合理分配各种资源，构建新系统的详细设计方案和相关模型，指导系统实施工作的顺利开展。

系统设计的主要内容包括概要设计和详细设计。概要设计又称为系统总体结构设计，它是系统开发过程中很关键的一步，其主要任务是将系统的功能需求分配给软件模块，确定每个模块的功能和调用关系，形成软件的模块结构图，即系统结构图。在概要设计中，将系统开发的总任务分解成许多个基本的、具体的任务，而为每个具体任务选择适当的技术手段和处理方法的过程称为详细设计。根据任务的不同，详细设计又可分为多种，例如，网络设计、代码设计、输入/输出设计、处理流程设计、数据存储设计、用户界面设计、安全性和可靠性设计等。

1. 网络设计

网络设计的主要任务是，根据系统的要求选择网络结构，按照系统结构的划分，安排网络 and 设备的分布，然后根据物理位置考虑网络布线和设备的部署，还要根据实际业务的要求划定各网络节点的权限、级别和管理方式等，选择相应的系统软件和管理软件。有关网络设计的详细知识见 4.4.2 小节。

2. 代码设计

代码是用数字或字符来表示各种客观实体。在系统开发过程中，进行代码设计的主要目的是确保代码的唯一化、规范化和系统化。在进行代码设计时，首先需要考虑系统的编码问题，编码问题的关键在于分类，有了科学的分类方式后，系统要建立编码规范就相对容易。准确的分类是工作标准化、系列化、合理化的基础和保证，目前最常用的分类方法概括起来有两种，一种是线分类法，另一种是面分类法。线分类法是目前使用最多的一种方法，尤其是在手工处

理的情况下几乎成了唯一的方法,其基本原理是:首先给定母项,母项下分若干子项,子项又可以分为更小的子项,最后落实到具体对象,分类的结果造成了一层套一层的线性关系。面分类法与线分类法不同,主要从面角度来考虑分类。

编码是分类问题的一种形式化描述,如果分类问题解决得较好,编码问题就变成了一个简单的用什么样的字符来表示的问题。目前,常用的编码方式包括顺序码(例如,用001表示北京、002表示长沙)、数字码(例如,用纯数字来表示居民身份证号码)、字符码(例如,汉语拼音和英文等)和混合码(以数字和字符混合形式编码)。

在进行代码设计时,首先需要确定编码对象,考察是否有标准代码(例如,国际标准、国家标准、部门标准或行业标准等),如果有相应的标准代码,则应该遵循这些标准代码;然后,需要确定代码的种类与类型,考虑代码的检错功能;最后,编写代码表。代码设计是一个科学管理的问题,应遵循唯一性、合理性、可扩展性、简单性、适用性、规范性和系统性等原则,设计出一个好的代码方案对于系统的开发工作来说是一件极为有利的事情。

3. 输入/输出设计

输入设计的目的是确保向系统输入数据的完整性、正确性和一致性,其主要内容包括确定输入数据内容、输入方式设计、输入格式设计和检验方式设计。输出设计的目的是确保系统输出数据的完整性、正确性和一致性,其主要内容包括确定输出的内容、选择输出设备与介质,以及确定输出格式等。

输入设计需要遵循以下原则:

(1) 输入数据最少原则。在满足需求的前提下尽量提供较少的数据输入,数据的输入量越少,出错的概率越低,花费的时间也越少。

(2) 简单性原则。输入过程应尽量简单,如以性别、出生日期等数据设计作为选择项。这样做一方面方便用户的使用,节省输入时间;另一方面可以降低出错的可能性。

(3) 尽早验证原则。对输入数据的检验尽量接近数据的输入点,及时发现输入中存在的错误,以便能够尽早进行改正。

(4) 少转换原则。输入数据尽量采用原始的数据格式,避免在数据转换过程中发生错误。

系统输出一般包括中间输出和最终输出,用户关心的是系统的最终输出。最常用的最终输出方式有两种,一种是报表输出,另一种是图形输出。一般来说,对于普通用户或具体数据的管理者和查阅者,应该以报表方式给出详细的数据记录;对于企业的高层领导和宏观或综合管理部门,则应该使用图形方式给出比例或综合发展趋势的信息,可以通过曲线图、柱状图、饼状图等图形方式来呈现。

4. 处理流程设计

处理流程设计是系统详细设计的重要组成部分,它的主要目的是确定各个系统模块的内部结构,即内部执行过程,包括局部数据组织和控制流,以及每个具体加工过程和实施细节。有关处理流程设计的详细知识将在13.2节中介绍。

5. 数据存储设计

数据存储设计主要是根据数据处理要求、处理方式、存储的信息量大小、数据使用的频率和所能提供的设备条件等，选择数据存储的方式、存储介质、数据组织方式和记录格式，并估算数据的容量。一个好的数据存储设计应该充分体现系统的业务流程，充分满足组织的各级管理要求。同时，还应该使得后续的系统开发工作方便、快捷，系统开销小，且易于管理和维护。

信息系统的主要目标是通过大量的数据获得管理所需要的信息，为了实现该目标，必须存储和管理大量的数据，因此，设计并建立一个良好的数据组织结构和数据库，使整个系统都可以迅速、方便、准确地存取和管理所需的数据，是衡量信息系统开发工作好坏的主要指标之一。数据存储设计直接影响到数据的存取效率、系统的实现效率和运行效率。数据存储设计主要包括两方面的工作：一是数据的统筹安排，例如，系统中设计多少个文件、数据文件如何分布、哪些数据是共享的、哪些是非共享的、哪些数据项应存放在一个文件中等；二是文件的数据结构设计，目前，大部分应用系统都使用关系数据库来存储数据，因此，数据结构设计的重点是关系数据库的设计，主要过程包括数据库的概念设计、逻辑设计和物理设计。有关数据库设计的详细知识见 5.4 节。

6. 用户界面设计

界面是系统与用户交互的最直接的层面，界面的好坏决定用户对系统的第一印象，而设计优良的界面能够引导用户自己完成相应的操作，起到向导的作用。同时，界面如同人的面孔，具有吸引用户的直接优势，设计合理的界面能给用户带来轻松愉悦的感受和成功的感觉。相反，由于界面设计的失败，让用户有挫败感，再实用强大的功能都可能在用户的畏惧与放弃中付诸东流。通常情况下，良好的用户界面设计需要遵循以下基本原则：

(1) 置于用户控制之下。在定义人机交互方式时，不强迫用户采用不必要的或者不情愿的方式来进行操作，允许交互的中断和撤销。当用户操作技能等级提高时，可以实现流水化的交互方式，允许用户定制交互方式，以便使用户界面与内部技术细节隔离，允许用户和出现在屏幕上的对象直接进行交互。

(2) 减轻用户的记忆负担。尽量减轻对用户记忆的要求，创建有意义的默认设置，定义一些符合用户直觉的访问途径，适当定义一些快捷方式，界面的视觉布局应该尽量与真实世界保持一致，并能够以不断扩展的方式呈现信息。用户可以快速学习并使用系统，尽量提供“傻瓜式”的操作界面，方便用户使用。界面中各个元素的名称应该易懂，用词准确，避免模棱两可的字眼，使用户能够做到“望文知意”，理想的情况是用户不用查阅帮助，就能知道该界面元素的功能，并正确地进行相关操作。

(3) 保持界面一致性。用户应以一致的方式提供或获取信息，所有可视信息的组织需要按照统一的设计标准，在系列化的应用软件中需要保持一致性，用户已经很熟悉的一些界面交互模型不到万不得已时，不要随意进行修改。需要确保用户界面操作和使用的一致性，例如，所有窗口按钮的位置要一致、提示信息和界面元素的命名要一致、界面颜色和风格要一致等。用户界面的一致性可以使用户能够统一地对待系统的各个不同的功能界面，以及系列化的系统，

从而降低培训和支持成本。

以上三条原则由著名用户界面设计专家 Theo Mandel 博士所创造，通常称之为“人机交互的“黄金三原则”。另外，在设计用户界面时，还需要保证界面的合理性和独特性，有效进行组合，注重美观与协调；恰到好处地提供快捷方式，注意资源协调等。

7. 安全性和可靠性设计

安全性和可靠性设计的目的是确保系统的安全性和可靠性，对系统的运行环境和数据处理进行有效控制，保证系统安全、有效地运行。其主要内容包括系统运行环境的安全性分析和控制，如对管理结构的组织、硬件和系统软件、自然环境等方面的分析与必要的监督和控制等；还包括对数据处理的控制，例如，对输入内容和输入方式的控制、错误程序与异常处理等。

13.2 处理流程设计

处理流程设计的任务是设计出系统所有模块以及它们之间的相互关系，并具体设计出每个模块内部的功能和处理过程，为开发人员提供详细的技术资料。每个信息系统都包含了一系列核心处理流程，例如，OA 系统的考勤流程、在线教育平台系统的组卷和考试流程、网上购物系统的购物和支付流程、航空订票系统的订票和退票流程等，对这些处理流程的理解和实现将直接影响系统的功能和性能。因此，系统设计人员需要认真面对处理流程的设计，深入理解系统的核心处理流程，通过对业务流程的设计来对现实世界进行建模，及时完善和调整系统分析和设计过程中的遗漏和不合理之处。

13.2.1 流程设计概述

系统处理流程对应于现实世界中的真实业务过程，通过对业务流程的设计，可以对其进行建模，以便使用信息系统来取代传统的手工处理，提高业务处理的效率和准确性，降低业务处理成本。

1. 流程

ISO 9000 定义业务流程（Business Process）为一组将输入转化为输出的相互关联或相互活动的活动。流程具有目标性、内在性、整体性、动态性、层次性和结构性等特点。一般来说，流程包括 6 个基本要素，分别是输入资源、活动、活动的相互作用（结构）、输出结果、用户和价值。例如，在线教育平台系统中的“开通课程”流程，其 6 个要素如表 13-1 所示。

表 13-1 开通课程流程的 6 个要素

要素名称	要素含义
输入资源	需要开通课程的注册用户名
活动	开通课程的业务逻辑（例如，用户名合法性判断、时间判断和费用计算等）
活动的相互作用	开通课程与其他活动（例如，在线测试等）流程的相互关系

(续表)

要素名称	要素含义
输出结果	开通课程成功后获取的短消息通知和电子邮件通知
用户	已交纳课程学习费用的注册用户
价值	用户可通过该流程实现学习课程的功能

流程的概念包括流程定义和流程实例，其中流程定义是指对业务过程的形式化表示，它定义了过程运行中的活动和所涉及的各种信息，这些信息包括过程的开始和完成条件、构成过程的活动，以及这些活动间的切换规则、用户需要完成的任务、可能被调用的应用、工作流的引用和数据的定义等；流程实例也称为工作，是一个流程定义的运行实例，即一次具体的流程操作，例如，一次在线测试过程、一次在线支付过程等。流程设计人员可以通过流程定义工具来定义流程，流程定义工具可以是独立的软件，也可以是 workflow 管理系统的一部分。

2. 工作流

根据工作流管理联盟（WorkFlow Management Coalition, WFMC）的定义，工作流是一类能够完全或者部分自动执行的业务过程，根据一系列过程规则、文档、信息或任务，在不同的执行者之间传递和执行。简单地说，工作流就是一系列相互衔接、自动进行的业务活动或任务，一个工作流包括一组活动（或任务）及它们的相互顺序关系，还包括流程和活动的启动和终止条件，以及对每个活动的描述。工作流可以部分或全部模拟现实世界中的信息传递。例如，在线教育平台系统中的开通课程流程，对应需要传递的信息就是课程申请单，员工填写好课程申请单后可将其发送给上级主管审批，主管审批后可以转交给培训部门备案，培训部门成功备案后可转交给财务部门，以便核算辅导老师工资。整个流程包括多个活动，不同的用户可以执行不同的活动，每个活动都有其启动和终止条件，例如，培训部门备案的启动条件是接收到已通过主管审批的课程申请单，而终止条件是成功记录课程信息，并将其转交给财务部门。

工作流管理是人与计算机共同工作的自动化协调、控制和通信，在信息化的业务过程中，通过在网络上运行相应的软件，使所有活动的执行都处于受控状态。在工作流管理下，可以对工作进行监控，并可以进行工作的指派。例如，如果将开通课程流程自动化，并构建一个软件模块来实现该功能，即可对开通课程工作流进行管理。

3. 活动及其所有者

活动是流程定义中的一个基本要素，一次活动可以改变流程处理数据的内容、流程的状态，并可能将流程推动到其他活动中去。活动可以由人来完成，也可以是系统自动进行处理，例如，通过时间等自动触发的活动。每个活动均有输入、处理和输出，例如，上级主管的审批活动，其输入是新创建的课程申请单，输出的是已通过审批的课程申请单或未通过审批的课程申请单。在输入和输出的转换过程中需要进行业务逻辑的判断，例如，课程类型为软考，则需要判断用户基础是否合适、时间是否合理，以及费用是否足够等。如果审批通过，则处理数据（课程申

请单)的状态发生改变,由新建课程申请单转变为已审批课程申请单,通常在工作流管理系统中可以标注不同状态的数据。

活动的所有者是流程参与者(包括人或其他系统)之一,他们有权决定该活动是否结束,当活动结束后,可以将活动推动到其他活动中(可能是下一个活动,也可能是前一个活动)。例如,在开通课程流程中,审批活动的参与者和所有者是部门主管,他们可以执行审批活动,如果课程申请单通过审批,可以将课程申请单传递给培训部门,以便执行下一个活动;也可以将课程申请单退还给咨询部门(创建课程申请单的员工),进行修改或者终止课程申请流程。

活动的所有者是整体控制流程实例执行过程的参与者,通常活动的所有者是流程的发起人,他们对流程的各项活动都很关注,而且可以整体控制流程实例的执行,例如,开通课程流程中的创建课程申请单的员工。

4. 工作项

工作项代表流程实例中活动的参与者将要执行的工作,例如,在开通课程流程中,某员工创建的一张课程申请单即为一个工作项。工作流管理系统包括若干个工作项,一个参与者也可对应多个工作项。通常,在系统实现时,不同的工作项有不同的编号,可以通过编号来快速定位到某一工作项。

13.2.2 工作流管理系统

工作流管理系统(WorkFlow Management System, WFMS)通过软件定义、创建工作流,并管理其执行。它运行在一个或多个工作流引擎上,这些引擎解释对过程的定义,并与工作流的参与者相互作用,根据需要调用其他IT工具或应用。例如,将考勤管理、内部信息交流、工作日报或周报处理等工作流管理模块集成在一个软件中,即可得到WFMS,这类WFMS即OA系统。

1. WFMS 的基本功能

WFMS将业务流程中工作如何组织与协调的规则抽象出来,在WFMS的协助下,开发人员遵从一定的编程接口和约定,就可以开发出更具灵活性的事务处理系统,用户无须重新开发即可更改工作流程,以适应业务的变更。WFMS的基本功能体现在以下几个方面:

(1) 对工作进行建模:即定义工作流,包括具体的活动和规则等,所创建的模型是同时可以被人和计算机所“理解”的,工作流对应现实世界的业务处理过程,不能改变真实业务的处理逻辑。

(2) 工作流执行:遵循工作流模型来创建和执行实际的工作流,即通过WFMS可以执行多个工作项。

(3) 业务过程的管理和分析:监控和管理执行中的业务(工作流),例如,进度完成情况和数据所处状态、工作分配与均衡情况等。

2. WFMS 的组成

工作流参考模型(Workflow Reference Model, WRM)包含6个基本模块,分别是工作流

执行服务、 workflow 引擎、流程定义工具、客户端应用、调用应用和管理监控工具。这 6 个模块被认为是 WFMS 最基本的组成部分，WRM 同时也包括了这些模块之间的接口标准，包括 workflow 定义交换接口（接口一）、workflow 客户端应用接口（接口二）、调用应用接口（接口三）、WMFS 互操作接口（接口四）和系统管理和监控接口（接口五），如图 13-1 所示。

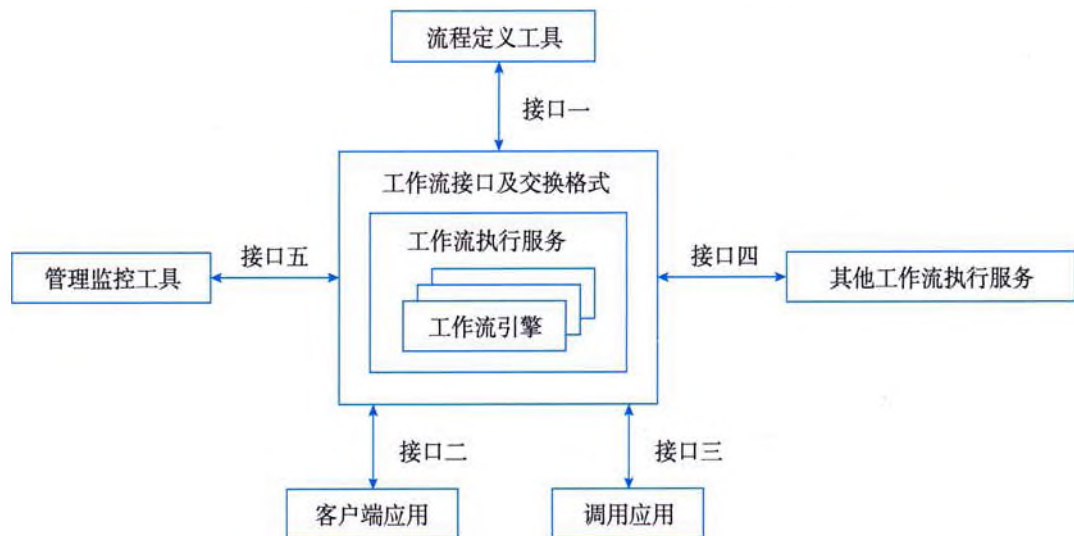


图 13-1 工作流参考模型

(1) 工作流执行服务。工作流执行服务是 WFMS 的核心模块，它的功能包括创建和管理流程定义，创建、管理和执行流程实例。在执行上述功能的同时，应用程序可能会通过编程接口与工作流执行服务交互，一个工作流执行服务可能包含有多个分布式工作的工作流引擎。该模块还为每个用户维护一个活动列表，告诉用户当前必须处理的任务，可以通过电子邮件或者短消息的形式提醒用户任务的到达，例如，在开通课程流程中，当新的课程申请到来时，可以提示上级主管。

(2) 工作流引擎。工作流引擎是为流程实例提供运行环境，并解释执行流程实例的软件模块，即负责流程处理的软件模块。

(3) 流程定义工具。流程定义工具是管理流程定义的工具，它可以通过图形方式把复杂的流程定义显示出来并加以操作。流程定义工具与工作流执行服务交互，一般该模块为设计人员提供图形化的用户界面。通过流程定义工具，设计人员可以创建新的流程或者改变现有流程，在流程定义时，可以指定各项活动的参与者的类型、活动之间的相互关系和传递规则等。

(4) 客户端应用。客户端应用是通过请求的方式与工作流执行服务交互的应用，也就是说，是客户端应用调用 workflow 来执行服务。客户端应用与工作流执行服务交互，它是面向最终用户的界面，可以将客户端应用设计为 B/S 架构或 C/S 架构。

(5) 调用应用。调用应用是被工作流执行服务调用的应用，调用应用与工作流执行服务交互。为了协作完成一个流程实例的执行，不同的工作流执行服务之间进行交互，调用应

用通常是 workflow 所携带数据的处理程序（常用的是电子文档的处理程序），它们在工作流执行过程中被调用，并向最终用户展示数据，这些应用程序的信息包括名称、调用方式和参数等。例如，在 OA 系统中，可以调用相关的程序来直接查看 Word 文档或者 Excel 表格数据等。

(6) 管理监控工具。管理监控工具主要指组织机构和参与者等数据的维护管理和流程执行情况的监控，管理监控工具与 workflow 执行服务交互。WFMS 通过管理监控工具提供对流程实例的状态查询、挂起、恢复和销毁等操作，同时提供系统参数和系统运行情况统计等数据。用户可以通过图形或者图表的方式对系统数据进行汇总与统计，并可随时撤销一些不合理的流程实例。

为了降低这 6 个模块的耦合度，使得模块之间相互独立，可通过接口来进行连接和调用。这 6 个模块之间可以通过以下 5 个接口进行交互：

(1) 工作流定义交换接口（接口一）。用于在流程定义工具与执行服务之间交换工作流定义，当工作流定义发生改变时，其处理流程将发生变化，执行服务也应该相应进行调整。

(2) 工作流客户端应用接口（接口二）。用于工作流客户端应用访问工作流引擎和工作列表，客户端应用是最终用户直接操作的界面，只要设计合理，可以实现多个不同的客户端应用调用同一个工作流引擎。

(3) 调用应用接口（接口三）。用于调用不同的应用系统，例如，在 OA 系统中调用 Doc 文档阅读器、PDF 文档阅读器或者计算器等。

(4) WMFS 互操作接口（接口四）。用于不同的 WMFS 之间的互操作，例如，在线教育平台系统可以提供学员成绩，而用户空间系统允许学员在线查询成绩，这两个系统之间有所关联，在某些功能的实现上提供了互操作。

(5) 系统管理和监控接口（接口五）。用于系统管理应用访问工作流执行服务。

WRM 为 WFMS 的关键模块提供了功能描述，并描述了关键模块之间的交互，而且这个描述是独立于特定产品或技术的实现的。从功能的角度定义 5 个关键模块的交互接口，推动了信息交换的标准化，使得不同产品间的互操作成为可能。

13.2.3 流程设计工具

在处理流程设计过程中，为了更清晰地表达过程规则说明，陆续出现了一些用于表示处理流程的工具，这些工具包括三类：图形工具、表格工具和语言工具。其中常见的图形工具包括程序流程图（Program Flow Diagram, PFD）、IPO 图、N-S 图（盒图）、问题分析图、判定树，表格工具包括判定表，语言工具包括过程设计语言等。

1. 程序流程图

程序流程图用一些图框表示各种操作，它独立于任何一种程序设计语言，比较直观、清晰，易于学习掌握。但它也存在一些严重的缺点，例如，程序流程图所使用的符号不够规范，常常会使用一些习惯性用法，特别是表示程序控制流程的箭头可以不受任何约束，随意转移控制，这些现象显然是与软件工程化的要求相背离的。为了消除这些缺点，应对流程图所使用的符号

做出严格定义，不允许人们随心所欲地画出各种不规范的流程图。为了更好地使用流程图描述结构化程序，必须对流程图进行限制。流程图中只能包括如图 13-2 所示的 5 种基本控制结构，任何复杂的程序流程图都应由这 5 种基本控制结构组合或嵌套而成。

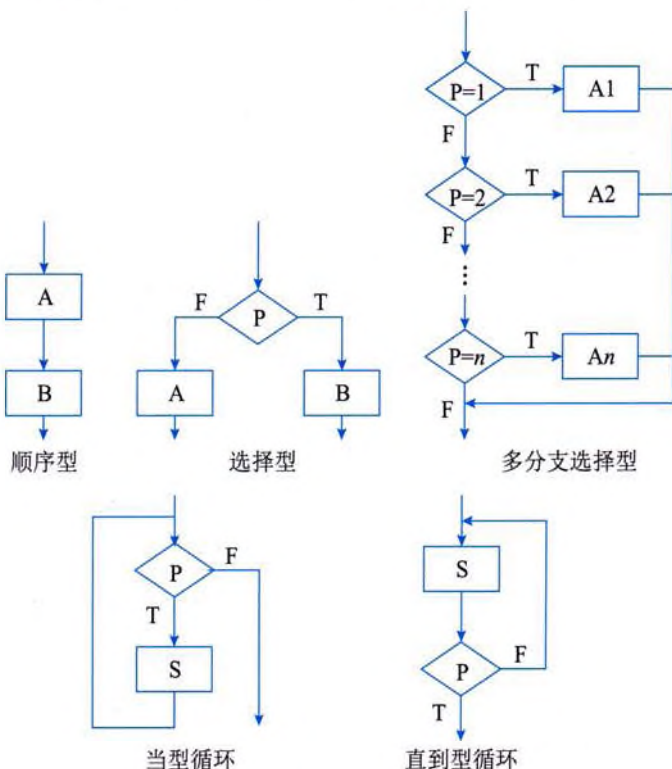


图 13-2 程序流程图的 5 种基本控制结构

2. IPO 图

IPO 图是由 IBM 公司发起并逐步完善的一种流程描述工具。系统分析阶段产生的数据流图经转换和优化后形成系统模块结构图的过程中将产生大量的模块，分析与设计人员应为每个模块写一份说明，此时即可用 IPO 图来对每个模块进行表述，IPO 图用来描述每个模块的输入、输出和数据加工。IPO 基本结构图的示例如图 13-3 所示。

IPO 图是系统设计中重要的文档资料之一，其主体是处理过程说明，可以采用流程图、判定树、判定表、盒图、问题分析图或过程设计语言来进行描述。IPO 图中的输入、输出与功能模块、文件及系统外部项都需要通过数据字典来描述，同时需要为其中的某些元素添加注释。

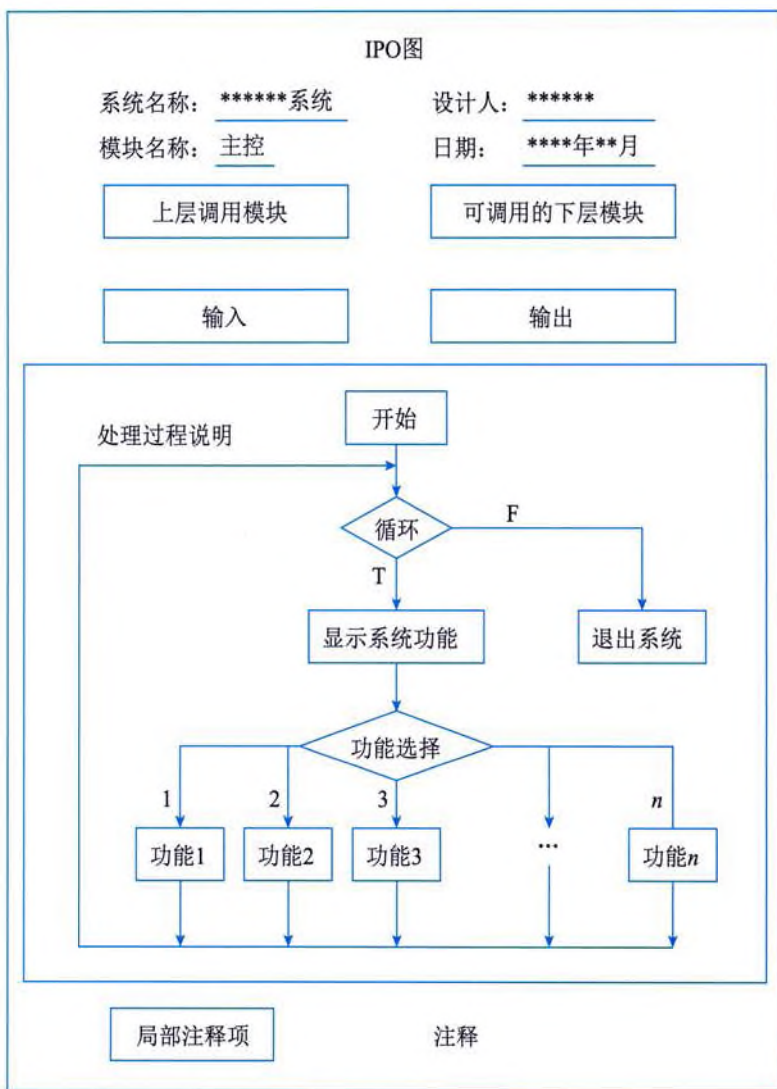


图 13-3 IPO 基本结构图

3. N-S 图

为避免流程图在描述程序逻辑时的随意性与灵活性，美国学者 I.Nassi 和 B.Shneiderman 在 1973 年提出了用方框代替传统的 PFD，通常把这种图称为 N-S 图或盒图。与 PFD 类似，在 N-S 图中也包括 5 种基本控制结构，分别是顺序型、选择型、当型循环（WHILE 循环型）、直到型循环（UNTIL 循环型）和多分支选择型，如图 13-4 所示，任何一个 N-S 图都是这 5 种基本控制结构相互组合与嵌套的结果。

在 N-S 图中，过程的作用域明确，N-S 图没有箭头，不能随意转移控制，而且容易表示嵌套关系和层次关系，并具有强烈的结构化特征。但是当问题很复杂时，N-S 图可能很大。

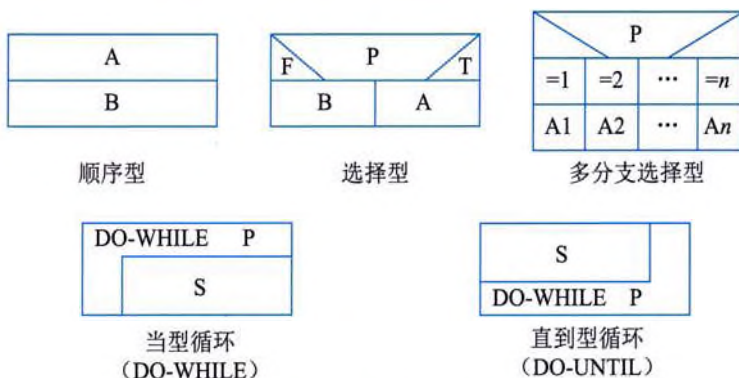


图 13-4 N-S 图的 5 种基本控制结构

4. 问题分析图

问题分析图 (Problem Analysis Diagram, PAD) 是继 PFD 和 N-S 图之后, 又一种描述详细设计的工具, 它由日立公司于 1979 年提出, 也是一种支持结构化程序设计的图形工具。PAD 也包含 5 种基本控制结构, 并允许递归使用, 如图 13-5 所示。

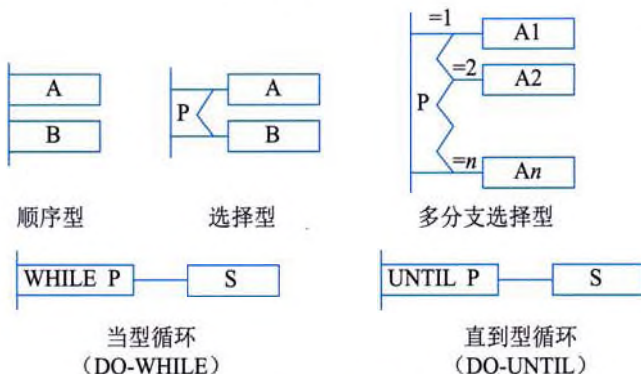


图 13-5 PAD 的 5 种基本控制结构

PAD 的执行顺序是从左侧主干线的上端节点开始, 自上而下依次执行; 每遇到判断或循环, 就自左而右进入下一层, 从表示下一层的纵线上端开始执行, 直到该纵线下端, 再返回上一层的纵线的转入处; 如此继续, 直到执行到主干线的下端为止。可以以 PAD 为基础, 按照一个机械的变换规则编写计算机程序。PAD 具有清晰的逻辑结构、标准化的图形, 更重要的是, 它引导设计人员使用结构化程序设计方法, 从而提高程序的质量。

5. 过程设计语言

过程设计语言 (Process Design Language, PDL) 也称为结构化语言或伪代码 (pseudo code), 它是一种混合语言, 采用自然语言的词汇和结构化程序设计语言的语法, 用于描述处理过程, 类似于编程语言。过程设计语言用于描述模块中算法和加工逻辑的具体细节, 以便在开发人员之间比较精确地进行交流。

过程设计语言的语法规则一般分为外层语法和内层语法。外层语法用于描述结构, 采用与

一般编程语言类似的关键字（例如，IF-THEN-ELSE、WHILE-DO 等），外层语法应当符合一般程序设计语言常用语句的语法规则；内层语法用于描述操作，可以采用自然语句（例如，英语和汉语等）中的一些简单的句子、短语和通用的数学符号来描述程序应执行的功能。

过程设计语言仅仅是对算法或加工逻辑的一种描述，是不可执行的。使用过程设计语言，可以做到逐步求精，从比较概括和抽象的过程设计语言程序开始，逐步写出更详细、更精确的描述，其写法比较灵活。它使用自然语言来描述处理过程，不必考虑语法错误，有利于设计人员把主要精力放在描述算法和加工逻辑上。

6. 判定表

对于具有多个互相联系的条件和可能产生多种结果的问题，用结构化语言描述显得不够直观和紧凑，这时可以用以清楚、简明为特征的判定表（Decision Table）来描述。判定表采用表格形式来表达逻辑判断问题，表格分成 4 个部分，左上部分为条件说明，左下部分为行动说明，右上部分为各种条件的组合说明，右下部分为各条件组合下相应的行动。在表的右上部分中列出所有条件，T 表示该条件取值为真，F 表示该条件取值为假，空白表示这个条件无论取何值对动作的选择并不产生影响，在判定表右下部分中列出所有的处理动作，Y 表示执行对应的动作，空白表示不执行该动作。判定表右半部分的每一列实质上是一条规则，规定了与特定条件取值组合相对应的动作。

例如，某批发公司本着薄利多销的原则制定了折扣策略，规定在与客户成交时，可根据不同情况对客户应交货款打一定折扣，表 13-2 为使用判定表描述的该公司的折扣策略。其中，C1～C3 为条件，A1～A4 为行动，1～8 为不同条件的组合，T 为条件满足，F 为不满足，Y 为该条件组合下的行动。例如，条件 4 表示若交易额在 50 000 元以上、最近 3 个月中有欠款且与本公司交易在 20 年以下，则可享受 5% 的折扣率。

表 13-2 判定表描述的某公司折扣策略

不同条件组合 条件和行动	1	2	3	4	5	6	7	8
C1: 每年交易额在 50 000 元以上	T	T	T	T	F	F	F	F
C2: 最近 3 个月无欠款	T	T	F	F	T	T	F	F
C3: 与本公司交易 20 年及以上	T	F	T	F	T	F	T	F
A1: 折扣率 15%	Y	Y						
A2: 折扣率 10%			Y					
A3: 折扣率 5%				Y				
A4: 无折扣率					Y	Y	Y	Y

7. 判定树

判定树（Decision Tree）也是用来表示逻辑判断问题的一种常用的图形工具，它用树来表达不同条件下的不同处理流程，比用语言、表格的方式更为直观。判定树的左侧（称为树根）为加工名，中间是各种条件，所有的行动都列于最右侧。例如，与表 13-2 对应的判定树如图 13-6 所示。

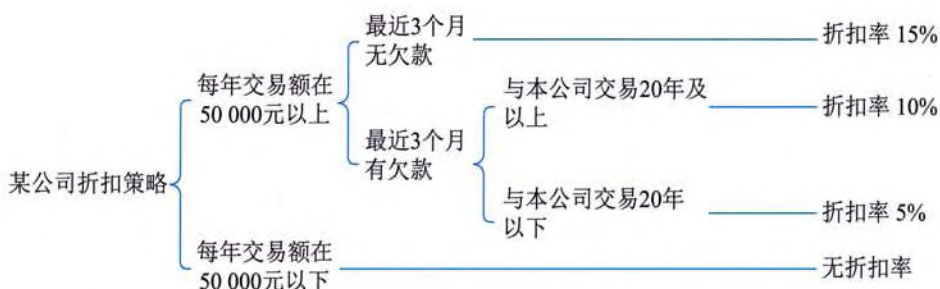


图 13-6 判定树描述的某公司折扣策略

13.3 结构化设计

结构化设计（Structured Design, SD）是一种面向数据流的方法，它以 SRS 和 SA 阶段所产生的数据流图和数据字典等文档为基础，是一个自顶向下、逐步求精和模块化的过程。SD 方法的基本思想是将软件设计成由相对独立且具有单一功能的模块组成的结构。软件设计可分为概要设计和详细设计两个阶段，其中概要设计的主要任务是确定软件系统的结构，对系统进行模块划分，确定每个模块的功能、接口和模块之间的调用关系；详细设计的主要任务是为每个模块设计实现的细节。

13.3.1 模块结构

系统是一个整体，它具有整体性的目标和功能，但这些目标和功能的实现又是由相互联系各个组成部分共同工作的结果。人们在解决复杂问题时使用的一个很重要的原则，就是将它分解成多个小问题分别处理。在处理过程中，需要根据系统总体要求，协调各业务部门的关系。在 SD 中，这种功能分解就是将系统划分为模块，模块是组成系统的基本单位，它的特点是可以自由组合、分解和变换，系统中任何一个处理功能都可以看成一个模块。

1. 信息隐蔽与抽象

信息隐蔽原则要求采用封装技术，将程序模块的实现细节（过程或数据）隐藏起来，对于不需要这些信息的其他模块来说是不能访问的，使模块接口尽量简单。按照信息隐蔽原则，系统中的模块应设计成“黑盒”，模块外部只能使用模块接口说明中给出的信息，例如，操作和数据类型等。模块之间相对独立，这样既易于实现，也易于理解和维护。

抽象原则要求抽取事物最基本的特性和行为，忽略非本质的细节，采用分层次抽象的方式可以控制软件开发过程的复杂性，有利于软件的可理解性和开发过程的管理。通常，抽象层次包括过程抽象、数据抽象和控制抽象。

2. 模块化

在 SD 方法中，模块是实现功能的基本单位，它一般具有功能、逻辑和状态三个基本属性，其中功能是指该模块“做什么”，逻辑用于描述模块内部“怎么做”，状态是指该模块使用时的环境和条件。在描述一个模块时，必须按模块的外部特性与内部特性分别描述。模块的外部特

性是指模块的模块名、参数表和给程序乃至整个系统造成的影响；模块的内部特性则是指完成其功能的程序代码和仅供该模块内部使用的数据。对于模块的外部环境（例如，需要调用这个模块的上级模块）来说，只需要了解这个模块的外部特性就足够了，不必了解它的内部特性。在软件设计阶段通常是先确定模块的外部特性，然后再确定它的内部特性。

在SD方法中，系统由多个逻辑上相对独立的模块组成，在模块划分时需要遵循如下原则：

（1）模块的大小要适中。系统分解时需要考虑模块的规模，过大的模块可能导致系统分解不充分，其内部可能包括不同类型的功能，需要进一步划分，尽量使各个模块的功能单一；过小的模块将导致系统的复杂度增加，模块之间的调用过于频繁，反而降低了模块的独立性。一般来说，当一个模块的大小使其实现代码约1~2页纸，或者其实现代码行数约50~200行时，这种规模的模块易于实现和维护。

（2）模块的扇入和扇出要合理。一个模块的扇出是指该模块直接调用的下级模块的个数。扇出大表示模块的复杂度高，需要控制和协调过多的下级模块。扇出过大一般是因为缺乏中间层次，应该适当增加中间层次的控制模块。扇出太小时可以把下级模块进一步分解成若干个子功能模块，或者合并到它的上级模块中去。一个模块的扇入是指直接调用该模块的上级模块的个数。扇入大表示模块的复用程度高。设计良好的软件结构通常顶层扇出比较大，中间扇出较少，底层模块则扇入大。一般来说，系统的平均扇入和扇出系数为3或4，不应该超过7，否则会增大出错的概率。

（3）深度和宽度适当。深度表示软件结构中模块的层数，如果层数过多，则应考虑有些模块设计是否过于简单，看能否适当合并。宽度是指软件结构中同一个层次上的模块总数的最大值，一般说来，宽度越大系统越复杂。对宽度影响最大的因素是模块的扇出。在系统设计时，需要权衡系统的深度和宽度，尽量降低系统的复杂性，减少实施过程的难度，提高开发和维护的效率。

3. 耦合

耦合表示模块之间联系的程度。紧密耦合表示模块之间联系非常强；松散耦合表示模块之间联系比较弱；非耦合则表示模块之间无任何联系，是完全独立的。模块的耦合类型通常分为7种，根据耦合度从低到高排序如表13-3所示。

表 13-3 模块的耦合类型

耦合类型	描 述
非直接耦合	两个模块之间没有直接关系，它们之间的联系完全是通过主模块的控制和调用来实现的
数据耦合	一组模块借助参数表传递简单数据
标记耦合	一组模块通过参数表传递记录信息（数据结构）
控制耦合	模块之间传递的信息中包含用于控制模块内部逻辑的信息
外部耦合	一组模块都访问同一全局简单变量而不是同一全局数据结构，而且不是通过参数表传递该全局变量的信息
公共耦合	多个模块都访问同一个公共数据环境，公共的数据环境可以是全局数据结构、共享的通信区、内存的公共覆盖区等
内容耦合	包括：一个模块直接访问另一个模块的内部数据；一个模块不通过正常入口转到另一个模块的内部；两个模块有一部分程序代码重叠；一个模块有多个入口

对于模块之间耦合的强度，主要依赖于一个模块对另一个模块的调用、一个模块向另一个模块传递的数据量、一个模块施加到另一个模块的控制的多少，以及模块之间接口的复杂程度。

4. 内聚

内聚表示模块内部各成分之间的联系程度，是从功能角度来度量模块内的联系，一个好的内聚模块应当恰好做目标单一的一件事情。模块的内聚类型通常也可以分为7种，根据内聚度从高到低排序如表13-4所示。

表 13-4 模块的内聚类型

内聚类型	描 述
功能内聚	完成一个单一功能，各个部分协同工作，缺一不可
顺序内聚	处理元素相关，而且必须顺序执行
通信内聚	所有处理元素集中在一个数据结构的区域上
过程内聚	处理元素相关，而且必须按特定的次序执行
瞬时内聚（时间内聚）	所包含的任务必须在同一时间间隔内执行
逻辑内聚	完成逻辑上相关的一组任务
偶然内聚（巧合内聚）	完成一组没有关系或松散关系的任务

一般说来，系统中各模块的内聚越高，则模块间的耦合就越低，但这种关系并不是绝对的。耦合低使得模块间尽可能相对独立，从而各模块可以单独开发和维护；内聚高使得模块的可理解性和维护性大大增强。因此，在模块的分解中应尽量减少模块的耦合，力求增加模块的内聚，遵循“高内聚、低耦合”的设计原则。

5. 模块类型

在系统结构图（Structure Chart, SC）中不能再分解的底层模块称为原子模块。如果一个系统的全部实际加工（数据计算或处理）都由底层的原子模块来完成，而其他所有非原子模块仅仅执行控制或协调功能，这样的系统就是完全因子分解的系统。如果 SC 是完全因子分解的，就是最好的系统。一般而言，在 SC 中存在4种类型的模块，如图13-7所示。

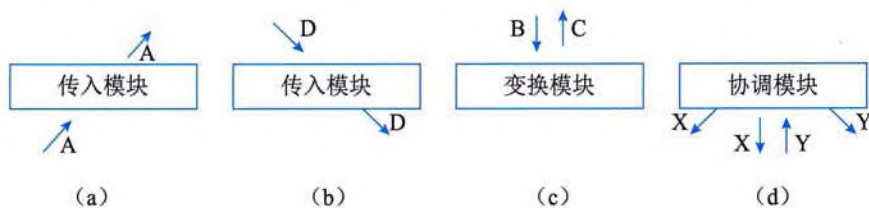


图 13-7 SC 的 4 种模块类型

(1) 传入模块。传入模块从下属模块中获取数据，经过某些处理，再将其传送给上级模块，如图13-7（a）所示。

(2) 传出模块。传出模块从上级模块中获取数据，进行某些处理，再将其传送给下属模块，如图13-7（b）所示。

(3) 变换模块。变换模块也称为加工模块，它从上级模块获取数据，进行特定的处理，然后转换成其他形式，再传送回上级模块，如图 13-7 (c) 所示。大多数计算模块（原子模块）都属于这一类。

(4) 协调模块。协调模块是对所有下属模块进行协调和管理的模块，如图 13-7 (d) 所示。在系统的 I/O 部分或数据加工部分可以找到这样的模块，在一个好的 SC 中，协调模块应在较高层出现。

在实际系统中，有些模块属于上述某一种类型，也有些模块是上述几种类型的组合。

13.3.2 系统结构图

系统结构图也称为模块结构图，它是软件概要设计阶段的工具，反映系统的功能实现和模块之间的联系与通信，包括各模块之间的层次结构，即反映了系统的总体结构。在系统分析阶段，系统分析师可以采用 SA 方法获取由 DFD、数据字典和加工说明等组成的系统的逻辑模型；在系统设计阶段，系统设计师可根据一些规则，从 DFD 中导出系统初始的 SC。

SC 的组成包括模块、模块之间的调用关系、模块之间的通信和辅助控制符号等 4 个部分。

(1) 模块。在 SC 中，模块用矩形框表示，框中标注模块的名字；对于已定义或者已开发的模块，可以用双纵边矩形框表示，如图 13-8 所示。



图 13-8 模块的图形表示

(2) 模块之间的调用关系。绘制方法是两个模块一上一下布局，以箭头相连，上面的模块是调用模块，箭头指向的模块是被调用的模块，例如图 13-9 中，“在线选课”模块调用“检索课程”模块。通常，箭头表示的连线可以用直线代替。

(3) 模块之间的通信。模块间的通信以表示调用关系的长箭头旁边的短箭头表示，短箭头的方向和名字分别表示调用模块和被调用模块之间信息的传递方向和内容。例如，在图 13-9 中，“在线选课”模块将“课程名、学期”信息传给“检索课程”模块，经加工处理后，“检索课程”模块将“课程号、课程名、选修人数、教师姓名”信息等再回传给“在线选课”模块。

(4) 辅助控制符号。当模块 A 有条件地调用模块 B 时，在箭头的起点标以菱形，模块 A 反复调用模块 C 时，在调用关系的连线上增加一个弧形的箭头，如图 13-10 所示。在 SC 中，条件调用时所依赖的判断条件和循环调用时所依赖的控制条件通常都无须注明。



图 13-9 模块之间的调用和通信的图形表示

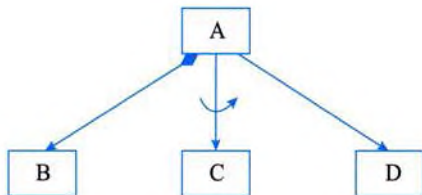


图 13-10 条件调用和循环调用的图形表示

常用的 SC 主要有变换型、事务型和混合型三种。

1. 变换型 SC

信息沿着输入通道进入系统，然后通过变换中心（也称为主加工）处理，再沿着输出通道离开系统，具有这一特性的信息流称为变换流。变换流对应的基本形态及其对应的 SC 如图 13-11 所示。

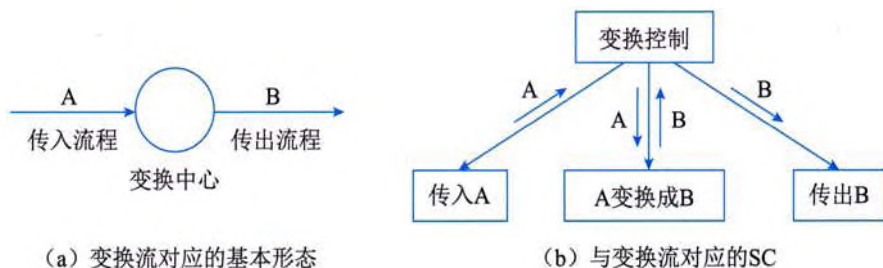


图 13-11 基本变换流及其对应的 SC

变换型 SC 可明显地分成输入、变换（主加工）和输出三大部分，它的功能是将输入的数据经过加工后输出，如图 13-12 所示。

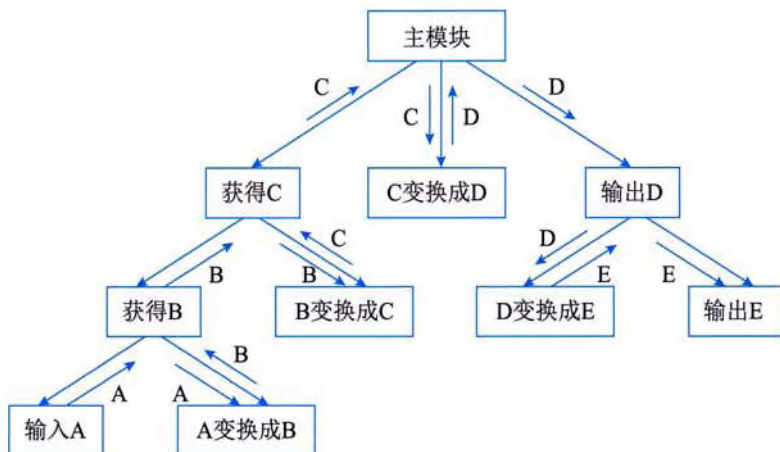


图 13-12 变换型 SC

变换型系统在工作时，首先主模块受到控制，然后控制沿着结构逐层到达底层的输入模块，当底层模块输入数据 A 后，A 由下至上逐层传送，逐步由物理输入 A 变成逻辑输入 C；接着，在主控模块控制下，C 经中心变换模块转换成逻辑输出 D，D 再由上至下逐层传送，逐步把逻辑输出 D 变成物理输出 E。这里的逻辑输入和逻辑输出分别为系统主处理的输入数据流和输出数据流，而物理输入和物理输出是指系统输入端和系统输出端的数据。

2. 事务型 SC

信息沿着输入通道到达一个事务中心，事务中心根据输入信息（即事务）的类型在若干个动作序列（称为活动流）中选择一个来执行，这种信息流称为事务流，如图 13-13 所示。

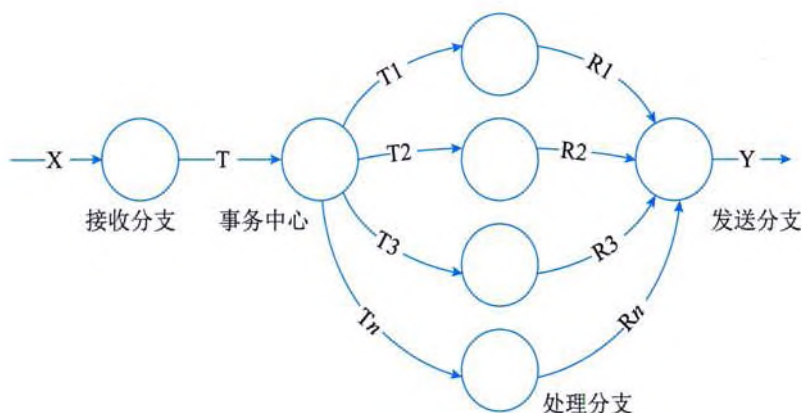


图 13-13 事务流图

由图 13-13 可以看出，事务流有明显的事务中心，各活动以事务中心为起点呈辐射状流出。事务型系统一般由三层组成，即事务层、操作层和细节层，它的功能是对接收的事务，按其类型选择某一类事务处理，如图 13-14 所示。

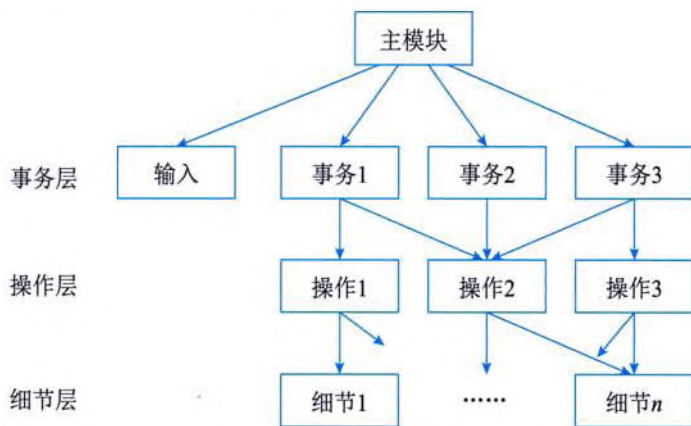


图 13-14 事务型 SC

在事务型 SC 中，主模块将按事务的类型选择调用某一事务处理模块，事务处理模块又调用若干个操作模块，而每个操作模块又调用若干个细节模块。各个事务处理模块是并列的，依赖于一定的选择条件，分别完成不同的事务处理工作。不同的事务处理模块可以共享一些操作模块。同样，不同的操作模块又可以共享一些细节模块。

3. 混合型 SC

在规模较大的系统中，其 DFD 往往是变换型和事务型的混合结构，如图 13-15 所示，此时，可把变换分析和事务分析应用在同一 DFD 的不同部分。例如，可以以变换分析为主、事务分析为辅进行设计，即先找出主处理，设计出结构图的上层，然后根据 DFD 各部分的结构特点，适当选用变换分析或事务分析就可得出 SC 的某个初始化方案。

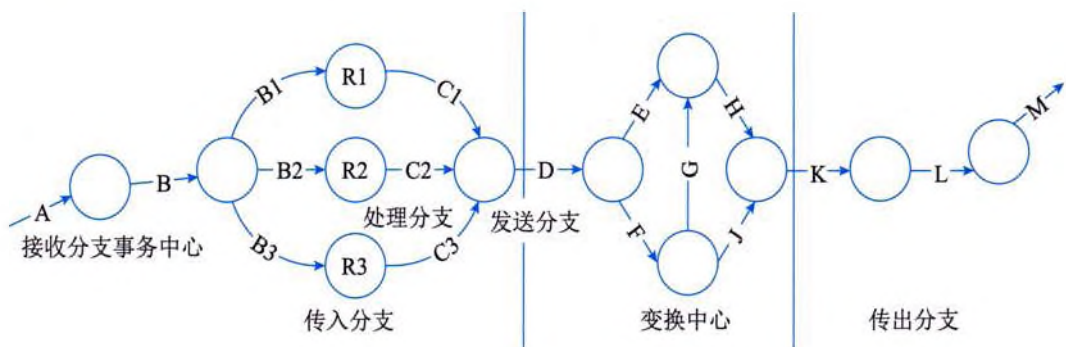
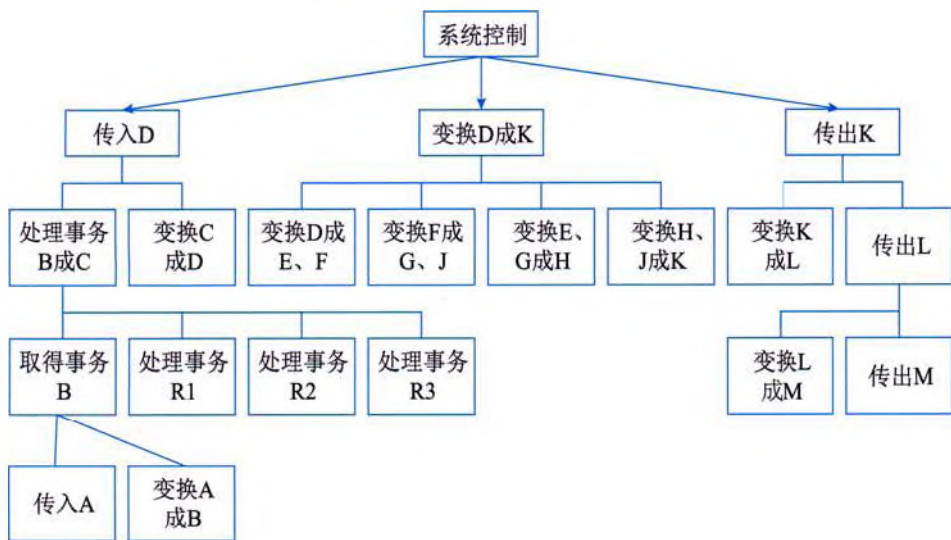


图 13-15 混合型 DFD

对于图 13-15 所示的混合型问题，从整体上可以将其看作一个从 A 到 M 的变换型问题：从 D 到 K 之间的变换是变换中心；从 A 到 D 是传入分支，具有事务型问题的特点；从 K 到 M 是传出分支。因此，该混合型问题结构图的上层可以由“传入 D”模块、“变换 D 成 K”模块和“传出 K”模块组成。“传入 D”模块的下层结构图由从传入分支映射得到的事务型问题结构图组成，“变换 D 成 K”模块和“传出 K”模块的下层结构图可以按通常的变换型问题映射方法获得。经过转换得到的混合型 SC 如图 13-16 所示。



13.4 面向对象设计

OOD 是 OOA 方法的延续，其基本思想包括抽象、封装和可扩展性，其中可扩展性主要通过继承和多态来实现。在 OOD 中，数据结构和在数据结构上定义的操作算法封装在一个对象之中。由于现实世界中的事物都可以抽象成对象的集合，所以 OOD 方法是一种更接近现实世界、更自然的系统设计方法。

13.4.1 设计软件类

类封装了信息和行为，是面向对象的重要组成部分，它是具有相同属性、方法和关系的对象集合的总称。在系统中，每个类都具有一定的职责，职责是指类所担任的任务。一个类可以有多种职责，设计得好的类一般至少有一种职责。在定义类时，将类的职责分解为类的属性和方法，其中属性用于封装数据，方法用于封装行为。设计类是 OOD 中最重要的组成部分，也是最复杂和最耗时的部分。

在系统设计过程中，类可以分为三种类型：实体类、控制类和边界类。

1. 实体类

实体类映射需求中的每个实体。实体类保存需要存储在永久存储体中的信息，例如，在线教育平台系统可以提取出学员类和课程类，它们都属于实体类。实体类通常都是永久性的，它们所具有的属性 and 关系是长期需要的，有时甚至在系统的整个生存期都需要。

实体类是对用户来说最有意义的类，通常采用业务领域术语命名，一般来说是一个名词。在用例模型向领域模型的转化中，一个参与者一般对应于实体类。通常可以从 SRS 中的那些与数据库表（需要持久存储）对应的名词着手来找寻实体类。通常情况下，实体类一定有属性，但不一定有操作。

2. 控制类

控制类是用于控制用例工作的类，一般是由动宾结构的短语（“动词 + 名词”或“名词 + 动词”）转化来的名词，例如，用例“身份验证”可以对应于一个控制类“身份验证器”，它提供了与身份验证相关的所有操作。控制类用于对一个或几个用例所特有的控制行为进行建模，控制对象（控制类的实例）通常控制其他对象，因此，它们的行为具有协调性。

控制类将用例的特有行为进行封装，控制对象的行为与特定用例的实现密切相关，当系统执行用例的时候，就产生了一个控制对象，控制对象经常在其对应的用例执行完毕后消亡。通常情况下，控制类没有属性，但一定有方法。

3. 边界类

边界类用于封装在用例内、外流动的信息或数据流。边界类位于系统与外界的交接处，包括所有窗体、报表、打印机和扫描仪等硬件的接口，以及与其他系统的接口。要寻找和定义边界类，可以检查用例模型，每个参与者和用例交互至少要有有一个边界类，边界类使参与者能与系统交互。边界类是一种用于对系统外部环境与其内部运作之间的交互进行建模的类。常见的边界类有窗口、通信协议、打印机接口、传感器和终端等。实际上，在系统设计时，产生的报表都可以作为边界类来处理。

边界类用于系统接口与系统外部进行交互，边界对象将系统与其外部环境的变更（例如，与其他系统的接口的变更、用户需求的变更等）分隔开，使这些变更不会对系统的其他部分造成影响。通常情况下，边界类可以既有属性也有方法。

13.4.2 对象持久化与数据库

在面向对象开发方法中，对象只能存在于内存中，而内存不能永久保存数据，如果要永久保存对象的状态，需要进行对象的持久化（persistence），对象持久化是把内存中的对象保存到数据库或可永久保存的存储设备中。在多层软件设计和开发中，为了降低系统的耦合度，一般会引入持久层（Persistence Layer），即专注于实现数据持久化应用领域的某个特定系统的一个逻辑层面，将数据用户和数据实体相关联，持久层的设计实现了数据处理层内部的业务逻辑和数据逻辑的解耦。

目前，关系数据库是使用最为广泛的数据库，如 DB2、Oracle、SQL Server 等。关系数据库中存放的是关系数据，即用二维表格表示的数据，它是非面向对象的。对象和关系数据其实是业务实体的两种表现形式，业务实体在内存中表现为对象，在数据库中表现为关系数据。内存中的对象之间存在关联和继承关系，而在数据库中，关系数据无法直接表达多对多关联和继承关系。因此，将对象持久化到关系数据库中，需要进行对象 / 关系的映射（Object/Relation Mapping，ORM），这是一项非常重要且烦琐耗时的工作。

在实际应用中，除了需要将内存中的对象持久化到数据库外，还需要将数据库中的关系数据再重新加载到内存中，以满足用户查询业务数据的需求。频繁地访问数据库，会对应用的性能造成很大影响，为了降低访问数据库的频率，可以将需要经常被访问的业务数据存放在缓存中，并且通过特定的机制来保证缓存中的数据与数据库中的数据同步。

数据持久化技术封装了数据访问细节，为大部分业务逻辑提供面向对象的 API。通过持久化技术，可以减少访问数据库数据的次数，增加应用程序执行速度；其代码重用性高，能够完成大部分数据库操作；松散耦合，使持久化不依赖于底层数据库和上层业务逻辑实现，更换数据库时只需修改配置文件而不用修改代码。随着对象持久化技术的发展，诞生了越来越多的持久化框架，目前，主流的持久化技术框架包括 CMP、Hibernate、iBatis 和 JDO 等。

1. CMP

在 J2EE 架构中，容器管理持久化（Container-Managed Persistence，CMP）是由 EJB 容器来管理实体 EJB 的持久化，EJB 容器封装了对象 / 关系的映射和数据访问细节。CMP 和 ORM 的相似之处在于，两者都提供对象 / 关系映射服务，都将对象持久化的任务从业务逻辑中分离出来，区别在于 CMP 负责持久化实体 EJB 组件，而 ORM 负责持久化简单的 Java 对象（Plain Ordinary Java Object，POJO），它是普通的基于 Java Bean 形式的实体域对象。

CMP 模式的优点在于它基于 EJB 技术，是 SUN J2EE 体系的核心部分，获得了业界的普遍支持，包括各大厂商和开源组织等。如果选择它来进行企业级开发，技术支持会非常完备，同时其功能日趋完善，包括了完善的事务支持、EJBQL 查询语言和透明的分布式访问等。CMP 的缺点在于开发的实体必须遵守复杂的 J2EE 规范，而 ORM 没有类似要求，其灵活性受到影响；而且，CMP 只能运行在 EJB 容器中，而普通 POJO 可以运行在任何一种 Java 环境中；尽管遵循 J2EE 的规范，但 EJB 的移植性比 ORM 要差。

2. Hibernate

Hibernate 和 iBatis 都是 ORM 解决方案,不同的是两者各有侧重。有关 Hibernate 的详细知识将在 16.3.2 小节中介绍。

3. iBatis

iBatis 提供 Java 对象到 SQL (面向参数和结果集) 的映射实现,实际的数据库操作需要通过手动编写 SQL 实现,与 Hibernate 相比,iBatis 最大的特点就是小巧,上手较快。如果不需要太多复杂的功能,iBatis 是既可满足要求又足够灵活的最简单的解决方案。

4. JDO

Java 数据对象 (Java Data Object, JDO) 是 SUN 公司制定的描述对象持久化语义的标准 API,它是 Java 对象持久化的新规范。JDO 提供了透明的对象存储,对开发人员来说,存储数据对象完全不需要额外的代码 (例如, JDBC API 的使用)。这些烦琐的例行工作已经转移到 JDO 产品提供商身上,使开发人员解脱出来,从而集中时间和精力在业务逻辑上。

另外,JDO 很灵活,因为它可以在任何数据底层上运行。JDBC 只能应用于关系型数据库,而 JDO 更通用,提供到任何数据底层的存储功能,包括关系型数据库、普通文件、XML 文件和对象数据库等,使得应用的可移植性更强。

13.4.3 面向对象设计的原则

对于 OO 系统的设计而言,在支持可维护性的同时,提高系统的可复用性是一个至关重要的问题,如何同时提高系统的可维护性和可复用性,是 OOD 需要解决的核心问题之一。在 OOD 中,可维护性的复用是以设计原则为基础的。常用的 OOD 原则包括开闭原则、里氏替换原则、依赖倒置原则、组合/聚合复用原则、接口隔离原则和最少知识原则等。这些设计原则都是面向复用的原则,遵循这些设计原则可以有效地提高系统的复用性,同时提高系统的可维护性。

1. 开闭原则

开闭原则是指软件实体应对扩展开放,而对修改关闭,即尽量在不修改原有代码的情况下进行扩展。此处的“实体”可以指一个软件模块、一个由多个类组成的局部结构或一个独立的类。

应用开闭原则可扩展已有的系统,并为之提供新的行为,以满足对软件的新需求,使变化中的系统具有一定的适应性和灵活性。对于已有的软件模块,特别是最重要的抽象层模块不能再修改,这就使变化中的系统有一定的稳定性和延续性,这样的系统同时满足了可复用性与可维护性。在 OOD 中,开闭原则一般通过在原有模块中添加抽象层 (例如,接口或抽象类) 来实现,它也是其他 OOD 原则的基础,而其他原则是实现开闭原则的具体措施。

2. 里氏替换原则

里氏替换原则由芭芭拉·利斯科夫 (Barbara Liskov) 提出,其基本思想是,一个软件实体如果使用的是一个基类对象,那么一定适用于其子类对象,而且觉察不出基类对象和子类对象

的区别，即把基类都替换成它的子类，程序的行为没有变化。反过来则不一定成立，如果一个软件实体使用的是一个子类对象，那么它不一定适用于基类对象。

在运用里氏替换原则时，尽量将一些需要扩展的类或者存在变化的类设计为抽象类或者接口，并将其作为基类，在程序中尽量使用基类对象进行编程。由于子类继承基类并实现其中的方法，程序运行时，子类对象可以替换基类对象，如果需要对类的行为进行修改，可以扩展基类，增加新的子类，而无须修改调用该基类对象的代码。

3. 依赖倒置原则

依赖倒置原则是指抽象不应该依赖于细节，细节应当依赖于抽象。换言之，要针对接口编程，而不是针对实现编程。在程序代码中传递参数时或在组合（或聚合）关系中，尽量引用层次高的抽象层类，即使用接口和抽象类进行变量类型声明、参数类型声明和方法返回类型声明，以及数据类型的转换等，而不要用具体类来做这些事情。为了确保该原则的应用，一个具体类应当只实现接口和抽象类中声明过的方法，而不要给出多余的方法，否则，将无法调用到在子类中增加的新方法。

实现开闭原则的关键是抽象化，并且从抽象化导出具体化实现，如果说开闭原则是 OOD 的目标的话，那么依赖倒置原则就是 OOD 的主要机制。有了抽象层，可以使系统具有很好的灵活性，在程序中尽量使用抽象层进行编程，而将具体类写在配置文件中，这样，如果系统行为发生变化，则只需要扩展抽象层，并修改配置文件，而无须修改原有系统的源代码，在不修改的情况下扩展系统功能，满足开闭原则的要求。依赖倒置原则是 COM、CORBA、EJB、Spring 等技术和框架背后的基本原则之一。

4. 组合 / 聚合复用原则

组合 / 聚合复用原则又称为合成复用原则，是在一个新的对象中通过组合关系或聚合关系来使用一些已有的对象，使之成为新对象的一部分，新对象通过委派调用已有对象的方法达到复用其已有功能的目的。简单地说，就是要尽量使用组合 / 聚合关系，少用继承。

在 OOD 中，可以通过两种基本方法在不同的环境中复用已有的设计和实现，即通过组合 / 聚合关系或通过继承。但首先应该考虑使用组合 / 聚合，组合 / 聚合可以使系统更加灵活，类与类之间的耦合度降低，一个类的变化对其他类造成的影响相对较少；其次才考虑继承，在使用继承时，需要严格遵循里氏替换原则，有效使用继承会有助于对问题的理解，降低复杂度，而滥用继承反而会增加系统构建和维护的难度，以及系统的复杂度。

通过继承来进行复用的主要问题在于继承复用会破坏系统的封装性，因为继承会将基类的实现细节暴露给子类，由于基类的内部细节通常对子类来说是透明的，所以这种复用是透明的复用，又称为白盒复用。如果基类发生改变，那么子类的实现也不得不发生改变；从基类继承而来的实现是静态的，不可能在运行时发生改变，没有足够的灵活性；而且继承只能在有限的环境中使用（例如，如果类没有声明就不能被继承）。

由于组合或聚合关系可以将已有的对象（也可称为成员对象）纳入到新对象中，使之成为新对象的一部分，新对象可以调用已有对象的功能，这样做可以使得成员对象的内部实现细节对于新对象是不可见的，因此，这种复用又称为黑盒复用。相对继承关系而言，其耦合度较低，

成员对象的变化对新对象的影响不大,可以在新对象中根据实际需要有选择性地调用成员对象的操作。组合/聚合复用可以在运行时动态进行,新对象可以动态地引用与成员对象类型相同的其他对象。

一般而言,如果两个类之间是 Has-A 的关系,则应使用组合或聚合;如果是 Is-A 关系,则可使用继承。Is-A 是严格的分类学意义上的定义,意思是一个类是另一个类的“一种”。而 Has-A 则不同,它表示某一个角色具有某一项责任。

5. 接口隔离原则

接口隔离原则是指使用多个专门的接口,而不使用单一的总接口。每个接口应该承担一种相对独立的角色,不多不少,不干不该干的事,该干的事都要干。这里的“接口”通常有两种不同的含义:一种是指一个类型所具有的方法特征的集合,仅仅是一种逻辑上的抽象;另一种是指某种语言具体的接口定义,有严格的定义和结构,例如,Java 语言中的 interface。对于这两种不同的含义,接口隔离原则的表达方式和含义都有所不同。

如果将“接口”理解成一个类型所提供的所有方法的特征集合,这就是一种逻辑上的概念,接口的划分将直接带来类型的划分。在这种情况下,可以将接口理解成角色,一个接口就只是代表一个角色,每个角色都有它特定的一个接口,此时,接口隔离原则可以称为角色隔离原则。

如果将“接口”理解成狭义的特定语言的接口,接口隔离原则表达的意思则是指接口仅提供客户端需要的行为,客户端不需要的行为则隐藏起来,应当为客户端提供尽可能小的单独的接口,而不要提供大的总接口。在面向对象编程语言中,如果需要一个接口,就需要实现该接口中定义的所有方法。因此,大的总接口使用起来不一定方便,为了使接口的职责单一,需要将大接口中的方法根据其职责不同,分别放在不同的小接口中,以确保每个接口使用起来都较为方便,并都承担单一角色。

6. 最少知识原则

最少知识原则也称为迪米特法则 (Law of Demeter),是指一个软件实体应当尽可能少地与其他实体发生相互作用。这样,当一个模块被修改时,就会尽量少地影响其他的模块,扩展会相对容易。这是对软件实体之间通信的限制,它要求限制软件实体之间通信的宽度和深度。

最少知识原则可分为狭义原则和广义原则。在狭义原则中,如果两个类之间不必彼此直接通信,那么这两个类就不应当发生直接的相互作用;如果其中的一个类需要调用另一个类的某一个方法,可以通过第三者转发这个调用。狭义原则可以降低类之间的耦合,但是会在系统中增加大量的小方法并散落在系统的各个角落。它可以使一个系统的局部设计简化,因为每个局部都不会和远距离的对象有直接的关联,但是也会造成系统的不同模块之间的通信效率降低,使得系统的不同模块之间不容易协调。

广义原则是指对对象之间的信息流量、流向和信息的影响的控制,主要是对信息隐藏的控制。信息的隐藏可以使各个子系统之间解耦,从而允许它们独立地被开发、优化、使用和修改,同时可以促进软件的复用。由于每个模块都不依赖于其他模块而存在,因此,每个模块都可以独立地在其他的地方使用。系统的规模越大,信息的隐藏就越重要,而信息隐藏的重要性也就越明显。

最少知识原则的主要用途在于控制信息的过载。在将最少知识原则运用到系统设计中时，要注意以下几点：

- (1) 在类的划分上，应当尽量创建松耦合的类，类之间的耦合度越低，就越有利于复用。一个处在松耦合中的类一旦被修改，不会对关联的类造成太大波动。
- (2) 在类的结构设计上，每个类都应当尽量降低其属性和方法的访问权限。
- (3) 在类的设计上，只要有可能，一个类型应当设计成不变类。
- (4) 在对其他类的引用上，一个对象对其他对象的引用应当降到最低。

13.5 设计模式

设计模式是前人经验的总结，它使人们可以方便地复用成功的设计和架构。当人们在特定的环境下遇到特定类型的问题，采用他人已使用过的一些成功的解决方案，一方面可以降低分析、设计和实现的难度，另一方面可以使系统具有更好的可复用性和灵活性。随着面向对象技术的发展和广泛应用，设计模式不再是一个新兴名词，它已逐步成为系统架构设计师、系统分析师、软件设计师和程序员所需掌握的基本技能之一。设计模式已广泛应用于面向对象系统的设计和开发，成为面向对象领域的一个重要组成部分。

13.5.1 设计模式概述

模式起源于建筑业而非软件业，模式之父——美国加利福尼亚大学环境结构中心研究所所长克里斯托弗·亚历山大（Christopher Alexander）博士用了约20年的时间，对舒适型住宅和周边环境进行了大量的调查和资料收集工作，发现人们对舒适型住宅和城市环境存在着共同的认知规律。他把这些规律归纳为253个模式，对每一个模式都从模式可适用的前提条件、在特定条件下要解决的目标问题和对目标问题的求解方案三个方面进行描述，并给出了从用户需求分析到建筑环境结构设计，直至经典实例的过程模型。亚历山大给出模式的经典定义如下：每个模式都描述了一个在实际环境中不断出现的问题，然后描述了该问题的解决方案的核心，通过这种方式，可以无数次地使用那些已有的解决方案，无须再重复相同的工作。也就是说，模式是在特定环境中解决问题的一种方案。

1. 软件模式与设计模式

软件模式是将模式的一般概念用于软件开发领域，即软件开发的总体指导思路或参照样板。最早将模式引入软件领域的是埃里克·伽玛（Erich Gamma）博士等四人组（Gang of Four, GoF），他们在1994年归纳、发表了23种设计模式，旨在用模式来统一沟通OO方法在分析、设计和实现之间的鸿沟。

软件模式包括设计模式、架构模式、分析模式和过程模式等，软件生存期的各个阶段都存在着被认同的模式。在软件模式领域，目前研究最为深入的是设计模式。设计模式是一套被反复使用、多数人知晓的、经过分类编目的、代码设计经验的总结，使用设计模式的目的是提高代码的可重用性，让代码更容易被他人理解，并保证代码可靠性。毫无疑问，这些设计模式已

经在前人的系统中得以证实并广泛使用,它使代码编写真正实现工程化,将已证实的技术表述成设计模式,也会使新系统开发者更加容易理解其设计思路。每种设计模式都是对13.4.3小节中某一种或多种OOD原则的体现。

2. 设计模式的关键元素

设计模式包含模式名称、问题、目的、解决方案、效果、实例代码和相关设计模式等基本要素,其中的关键元素包括以下4个方面:

(1) 模式名称。给模式取一个助记名,用一两个词语来描述模式待解决的问题、解决方案和使用效果,以便更好地理解模式并方便开发人员之间的交流。

(2) 问题。描述应该在何时使用模式,即在解决何种问题时可使用该模式。有时,在问题部分会包括使用模式必须满足的一系列先决条件。

(3) 解决方案。描述设计的组成成分、它们之间的相互关系及各自的职责和协作方式。模式就像一个模板,可应用于多种不同场合,所以解决方案并不描述一个特定而具体的设计或实现,而是提供一个问题的抽象描述和具有一般意义的元素组合(类或对象组合)。

(4) 效果。描述模式应用的效果以及使用模式时应权衡的问题,即模式的优缺点。没有一种解决方案是完美的,每种设计模式都具有自己的优点,但也存在一些缺陷,它们对于评价设计选择和理解使用模式的代价和好处具有重要意义。模式效果有助于选择合适的模式,它不仅包括时间和空间的权衡,还包括对系统的灵活性、可扩展性或可移植性的影响。

13.5.2 设计模式分类

根据目的和用途不同,设计模式可分为创建型(Creational)模式、结构型(Structural)模式和行为型(Behavioral)模式三种。创建型模式主要用于创建对象,结构型模式主要用于处理类或对象的组合,行为型模式主要用于描述类或对象的交互以及职责的分配。

根据处理范围不同,设计模式可分为类模式和对象模式。类模式处理类和子类之间的关系,这些关系通过继承建立,在编译时刻就被确定下来,属于静态关系。对象模式处理对象之间的关系,这些关系在运行时刻变化,更具动态性。

1. 创建型模式

创建型模式对类的实例化过程(即对象的创建过程)进行了抽象,能够使软件模块做到与对象的创建和组织无关。创建型模式隐藏了对象是如何被创建和组合在一起的,以达到使整个系统独立的目的。创建型模式包括工厂方法模式、抽象工厂模式、原型模式、单例模式和建造者模式等。

(1) 工厂方法(Factory Method)模式。工厂方法模式又称为虚拟构造器(Virtual Constructor)模式或多态模式,属于类的创建型模式。在工厂方法模式中,父类负责定义创建对象的公共接口,而子类则负责生成具体的对象,这样做的目的是将类的实例化操作延迟到子类中完成,即由子类来决定究竟应该实例化(创建)哪一个类。

(2) 抽象工厂(Abstract Factory)模式。抽象工厂模式又称为Kit模式,属于对象创建型模式。抽象工厂模式是所有形式的工厂模式中最为抽象和最具一般性的一种形态,它提供了一个

创建一系列相关或相互依赖对象的接口，而无须指定它们具体的类。在抽象工厂模式中，引入了产品等级结构和产品族的概念。产品等级结构是指抽象产品与具体产品所构成的继承层次关系；产品族是同一个工厂所生产的一系列产品，即位于不同产品等级结构且功能相关联的产品组成的家族。当抽象工厂模式退化到只有一个产品等级结构时，即变成了工厂方法模式。

(3) 原型 (Prototype) 模式。在系统开发过程中，有时候有些对象需要被频繁创建，原型模式通过给出一个原型对象来指明所要创建的对象类型，然后通过复制这个原型对象的办法，创建出更多同类型的对象。原型模式是一种对象创建型模式，用原型实例指定创建对象的种类，并且通过复制这些原型创建新的对象。原型模式又可分为两种：浅克隆和深克隆。浅克隆仅仅复制所考虑的对象，而不复制它所引用的对象，也就是其中的成员对象并不复制；深克隆除了对象本身被复制外，对象包含的引用也被复制，即成员对象也被复制。

(4) 单例 (Singleton) 模式。单例模式确保某一个类只有一个实例，而且自行实例化并向整个系统提供这个实例，这个类称为单例类，它提供全局访问的方法。

(5) 建造者 (Builder) 模式。建造者模式强调将一个复杂对象的构建与它的表示分离，使得同样的构建过程可以创建不同的表示。建造者模式是一步一步地创建一个复杂的对象，它允许用户只通过指定复杂对象的类型和内容就可以构建它们，用户不需要知道内部的具体构建细节。建造者模式属于对象创建型模式。

2. 结构型模式

结构型模式描述如何将类或对象结合在一起形成更大的结构。结构型模式描述两种不同的事物，即类与类的实例（对象），据此可以将结构型模式分为类结构型模式和对象结构型模式。结构型模式包括适配器 (Adapter) 模式、桥接 (Bridge) 模式、组合 (Composite) 模式、装饰 (Decorator) 模式、外观 (Facade) 模式、享元 (Flyweight) 模式和代理 (Proxy) 模式等。

(1) 适配器模式。适配器模式将一个接口转换成客户希望的另一个接口，从而使接口不兼容的那些类可以一起工作。适配器模式既可以作为类结构型模式，也可以作为对象结构型模式。在类适配器模式中，通过使用一个具体类将适配器适配到目标接口中；在对象适配器模式中，一个适配器可以将多个不同的适配者适配到同一个目标。

(2) 桥接模式。桥接模式将抽象部分与它的实现部分分离，使它们都可以独立地变化。它是一种对象结构型模式，又称为柄体 (Handle and Body) 模式或接口 (Interface) 模式。桥接模式类似于多重继承方案，但是多重继承方案往往违背了类的单一职责原则，其复用性比较差，桥接模式是比多重继承方案更好的解决方法。

(3) 组合模式。组合模式又称为整体 - 部分 (Part-Whole) 模式，属于对象的结构模式。在组合模式中，通过组合多个对象形成树形结构以表示整体 - 部分的结构层次。组合模式对单个对象（即叶子对象）和组合对象（即容器对象）的使用具有一致性。

(4) 装饰模式。装饰模式是一种对象结构型模式，可动态地给一个对象增加一些额外的职责，就增加对象功能来说，装饰模式比生成子类实现更为灵活。通过装饰模式，可以在不影响其他对象的情况下，以动态、透明的方式给单个对象添加职责；当需要动态地给一个对象增加功能，这些功能可以再动态地被撤销时，可使用装饰模式；当不能采用生成子类的方法进行扩

充时也可以使用装饰模式。

(5) 外观模式。外观模式是对象的结构模式，要求外部与一个子系统的通信必须通过一个统一的外观对象进行，为子系统中的一组接口提供一个一致的界面，外观模式定义了一个高层接口，这个接口使得这一子系统更加容易使用。

(6) 享元模式。享元模式是一种对象结构型模式，通过运用共享技术，有效地支持大量细粒度的对象。系统只使用少量的对象，而这些对象都很相似，状态变化很小，对象使用次数增多。享元对象能做到共享的关键是区分内部状态和外部状态。内部状态存储在享元对象内部并且不会随环境改变而改变，因此内部状态可以共享；外部状态是随环境改变而改变的、不可以共享的状态，享元对象的外部状态必须由客户端保存，并在享元对象被创建之后，在需要使用的时候再传入到享元对象内部，外部状态之间是相互独立的。

(7) 代理模式。代理模式是一种对象结构型模式，可为某个对象提供一个代理，并由代理对象控制对原对象的引用。代理模式能够协调调用者和被调用者，能够在一定程度上降低系统的耦合度，其缺点是请求的处理速度会变慢，并且实现代理模式需要额外的工作。

3. 行为型模式

行为型模式是对在不同的对象之间划分责任和算法的抽象化，它不仅仅是关于类和对象的，而且是关于它们之间的相互作用的。行为型模式分为类行为模式和对象行为模式两种，其中类行为模式使用继承关系在几个类之间分配行为，而对象行为模式则使用对象的聚合来分配行为。行为型模式包括职责链（Chain of Responsibility）模式、命令（Command）模式、解释器（Interpreter）模式、迭代器（Iterator）模式、中介者（Mediator）模式、备忘录（Memento）模式、观察者（Observer）模式、状态（State）模式、策略（Strategy）模式、模板方法（Template Method）模式、访问者（Visitor）模式等。

(1) 职责链模式。职责链模式是一种对象的行为型模式，避免请求发送者与接收者耦合在一起，让多个对象都有可能接收请求，将这些对象连接成一条链，并且沿着这条链传递请求，直到有对象处理它为止。职责链模式不保证每个请求都被接受，如果有一个请求没有明确的接收者，那么就不能保证它一定会被处理。

(2) 命令模式。命令模式是一种对象的行为型模式，类似于传统程序设计方法中的回调机制，它将一个请求封装为一个对象，从而可用不同的请求对客户进行参数化；对请求排队或者记录请求日志，以及支持可撤销的操作。命令模式是对命令的封装，将发出命令的责任和执行命令的责任分割开，委派给不同的对象，以实现发送者和接收者完全解耦，提供更大的灵活性和可扩展性。

(3) 解释器模式。解释器模式属于类的行为型模式，描述了如何为语言定义一个文法，如何在该语言中表示一个句子，以及如何解释这些句子，这里的“语言”是使用规定格式和语法的代码。解释器模式主要用在编译器中，在应用系统开发中很少用到。

(4) 迭代器模式。迭代器模式是一种对象的行为型模式，提供了一种方法来访问聚合对象，而不用暴露这个对象的内部表示。迭代器模式支持以不同的方式遍历一个聚合对象，复杂的聚合可用多种方法来进行遍历；允许在同一个聚合上可以有多个遍历，每个迭代器保持它自己的

遍历状态，因此，可以同时进行多个遍历操作。

(5) 中介者模式。中介者模式是一种对象的行为型模式，通过一个中介对象来封装一系列的对象交互。中介者使得各对象不需要显式地相互引用，从而使其耦合松散，而且可以独立地改变它们之间的交互。中介者对象的存在保证了对象结构上的稳定，也就是说，系统的结构不会因为新对象的引入带来大量的修改工作。

(6) 备忘录模式。备忘录模式确保在不破坏封装的前提下，捕获一个对象的内部状态，并在该对象之外保存这个状态，这样可以在以后将对象恢复到原先保存的状态。备忘录模式提供了一种状态恢复的实现机制，使得用户可以方便地回到一个特定的历史步骤。

(7) 观察者模式。观察者模式又称为发布 - 订阅模式、模型 - 视图模式、源 - 监听器模式或从属者 (Dependents) 模式，是一种对象的行为型模式。它定义了对象之间的一种一对多的依赖关系，使得每当一个对象状态发生改变时，其相关依赖对象都得到通知并被自动更新。观察者模式的优点在于实现了表示层和数据层的分离，并定义了稳定的更新消息传递机制，类别清晰，抽象了更新接口，使得相同的数据层可以有各种不同的表示层。

(8) 状态模式。状态模式是一种对象的行为型模式，允许一个对象在其内部状态改变时改变它的行为，对象看起来似乎修改了它的类。状态模式封装了状态的转换过程，但是它需要枚举可能的状态，因此，需要事先确定状态种类，这也导致在状态模式中增加新的状态类时将违反开闭原则，新的状态类的引入将需要修改与之能够进行转换的其他状态类的代码。状态模式的使用必然会增加系统类和对象的个数。

(9) 策略模式。策略模式是一种对象的行为型模式，定义一系列算法，并将每一个算法封装起来，并让它们可以相互替换。策略模式让算法独立于使用它的客户而变化，其目的是将行为和环境分隔，当出现新的行为时，只需要实现新的策略类。

(10) 模板方法模式。模板方法模式是一种类的行为型模式，用于定义一个操作中算法的骨架，而将一些步骤延迟到子类中。模板方法模式使得子类可以不改变一个算法的结构即可重定义该算法的某些特定步骤，其缺点是对于不同的实现，都需要定义一个子类，这会导致类的个数增加，但是更加符合类职责的分配原则，使得类的内聚性得以提高。

(11) 访问者模式。访问者模式是一种对象的行为型模式，用于表示一个作用于某对象结构中的各元素的操作，它使得用户可以在不改变各元素的类的前提下定义作用于这些元素的新操作。访问者模式使得增加新的操作变得很容易，但在一定程度上破坏了封装性。

13.6 输入 / 输出原型设计

系统输入 / 输出设计是与用户息息相关的，输入 / 输出设计对于系统的开发人员以及用户极为重要。系统输入 / 输出设计可以帮助开发人员更好地理解系统的需求和功能，从而确保系统的正确性和稳定性；可以使系统的代码更加清晰和易于维护，从而降低维护成本和风险；使系统更加灵活和可扩展，从而满足不同用户的需求和变化的市场需求；使系统更加易于使用和友好，从而提高用户的满意度和忠诚度；可以使开发人员更加高效地开发系统，从而降低开发成本和时间。

13.6.1 输入设计

系统输入设计是指在设计系统时，如何收集和处理用户输入的过程。以下是系统输入设计的一些关键考虑因素：

- (1) 用户需求：了解用户的需求和期望，以便设计出符合用户期望的输入方式。
- (2) 输入方式：选择适合用户的输入方式，例如使用键盘、鼠标、触摸屏、语音识别等。
- (3) 输入格式：确定输入数据的格式，例如文本、数字、日期、时间等。
- (4) 输入验证：对输入数据进行验证，确保数据的正确性和完整性。
- (5) 输入反馈：提供及时的输入反馈，例如错误提示、自动完成等。
- (6) 输入历史：记录用户的输入历史，以便用户快速输入相似的数据。
- (7) 输入安全：确保输入数据的安全性，例如防止恶意输入、保护用户隐私等。
- (8) 输入效率：优化输入流程，提高用户输入效率，例如自动填充、快捷键等。

综上所述，系统的输入设计是系统设计中非常重要的一环，它直接关系到系统的功能实现和用户体验。所以，在设计系统输入时，应当遵循以下7种设计原则：

1. 确定输入数据的类型和格式

在系统设计时，首先需要确定输入数据的类型和格式。在设计系统的输入时，首先需要确定输入数据的类型和格式。根据系统的需求，输入数据可以是文本、数字、图像、音频等。例如，电商网站的输入数据可能包括商品名称、价格、描述和图片等。

图13-17是用户在商城购物下单并购买商品的时序图，由于用户操作界面允许输入多种类型的数据，因此在确定输入数据类型的同时，还需要确定输入数据的格式。例如，文本数据可以是纯文本、富文本、Markdown等格式；数字数据可以是整数、浮点数、百分数等格式；图像数据可以是JPEG、PNG、GIF等格式；音频数据可以是MP3、WAV等格式。

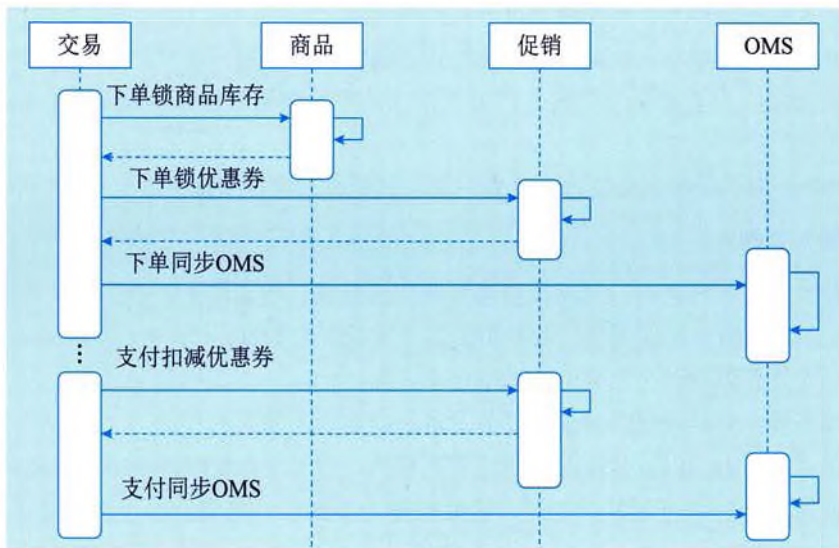


图 13-17 商城下单时序图

2. 确定输入数据的来源

确定输入数据的来源是设计系统输入的另一个重要方面。输入数据可以来自用户输入、传感器采集、外部 API 调用等。例如，一个智能家居系统的输入数据可能来自温度传感器、湿度传感器、光线传感器等。

3. 设计良好的输入界面

在确定输入数据来源时，还需要考虑输入数据的实时性和准确性。例如，传感器采集的数据可能需要实时处理，而用户输入的数据可能需要进行验证和清洗。

设计输入界面是系统输入设计的重要方面之一。输入界面需要根据系统的需求和用户体验进行设计。例如，在线购物网站的输入界面可能包括商品搜索框、商品分类下拉菜单、购物车等。

在设计输入界面时，需要考虑以下几个方面：

- (1) 界面布局：需要简洁明了，易于用户操作。
- (2) 控件选择：根据输入数据的类型和格式，选择合适的控件，例如文本框、下拉菜单、单选框、复选框等。
- (3) 输入提示：为了方便用户输入，需要提供输入提示，例如自动补全、下拉提示等。
- (4) 错误提示：为了避免用户输入错误数据，需要提供错误提示，例如数据格式错误、数据范围错误等。

4. 设计系统输入验证机制

为了保证输入数据的准确性和完整性，需要设计输入验证机制。输入验证机制可以对输入数据进行数据类型验证、数据格式验证、数据范围验证等。例如，一个电商网站的输入验证机制可能包括价格必须为数字、商品名称不能超过 50 个字符等。

在设计输入验证机制时，需要考虑以下几个方面：

- (1) 验证规则：根据输入数据的类型和格式，确定验证规则。
- (2) 错误提示：为了避免用户输入错误数据，需要提供错误提示，例如数据格式错误、数据范围错误等。
- (3) 安全性：为了避免恶意攻击，需要对输入数据进行安全验证。

5. 设计输入处理流程

在输入数据进入系统之前，可能需要进行数据预处理、数据清洗、数据转换等操作。例如，智能家居系统的输入数据可能需要进行数据清洗，去除异常数据。

在设计输入处理流程时，需要考虑以下几个方面：

- (1) 处理流程：根据系统的需求和业务逻辑，确定输入处理流程。
- (2) 数据清洗：为了保证数据的准确性和完整性，需要进行数据清洗，去除异常数据。
- (3) 数据转换：为了方便系统处理，需要对输入数据进行数据转换，例如将文本数据转换为数字数据。

6. 设计输入存储方案

输入数据可能需要存储到数据库、文件系统、云存储等。在设计输入存储方案时,需要考虑以下几个方面:

(1) 存储方式: 根据系统的需求和数据量大小, 确定存储方式, 例如数据库、文件系统、云存储等。

(2) 存储结构: 根据系统的需求和数据结构, 确定存储结构, 例如关系型数据库、非关系型数据库等。

(3) 存储安全: 为了保护输入数据的安全性, 需要对存储数据进行加密、访问控制等操作。

7. 设计输入安全机制

为了保护输入数据的安全性, 需要设计输入安全机制。输入安全机制可以对输入数据进行加密、访问控制、防止 SQL 注入等操作。例如, 一个在线银行系统的输入安全机制可能包括用户身份验证、数据加密等。

在设计输入安全机制时, 需要考虑以下几个方面:

(1) 访问控制: 为了保护输入数据的安全性, 需要对访问者进行身份验证和访问控制。

(2) 数据加密: 为了保护输入数据的机密性, 需要对输入数据进行加密。

(3) 防止 SQL 注入: 为了避免恶意攻击, 需要对输入数据进行防止 SQL 注入等操作。

综上所述, 系统的输入设计需要考虑输入数据的类型和格式、输入数据的来源、输入界面的设计、输入验证机制的设计、输入处理流程的设计、输入存储方案的设计、输入安全机制的设计等方面。在设计输入时, 需要根据系统的需求和实际情况进行综合考虑。

13.6.2 输出设计

系统输出设计可以帮助开发人员和用户在生活和工作中更好地把握方向和目标, 提高工作效率。系统的输出设计主要包括以下三个方面的内容:

(1) 确定输出内容: 确定用户在使用信息方面的需求, 然后设计输出信息的内容、信息形式(包括文字、图形、表格)、输出项目及数据结构、数据类型、位数及取值范围、数据的生成途径、完整性以及数据一致性等。

(2) 选择输出设备与介质: 常见的输出设备有显示终端、打印机、绘图仪、磁带机、多媒体设备; 输出介质有纸张、磁带、磁盘、胶卷、光盘等。

(3) 确定输出格式: 输出格式要满足使用者的要求和习惯, 做到格式清晰、美观、容易阅读和理解等。最终输出形式包括报表输出、图形输出等, 其输出形式需要根据系统分析和管理业务的需求来决定。

例如, 图 13-18 所示为某厂能耗综述, 以报表的形式向用户展示, 能够保证用户在观看体验上感觉更直观、更简洁。

一般情况下, 在进行输出设计时, 针对企业具体业务人员比较适合采用报表的形式, 针对企业高层领导、综合管理部门则适合用图形、图表方式来表示发展趋势, 供企业领导决策使用。



选择年份: 2016年 ▾

总耗水 (吨) 34422 同比+1.5%

总耗电 (万kW·h) 7275.00 同比+4.1%

总燃气消耗 (万m³) 1668.77 同比-0.98%

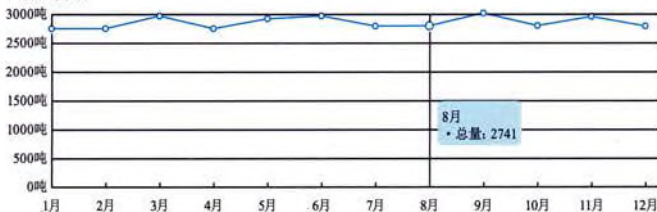
综合能耗 (kg标煤) 9668.77 同比-0.98%

处理气量: 1.7亿m³ 同比+0.1%

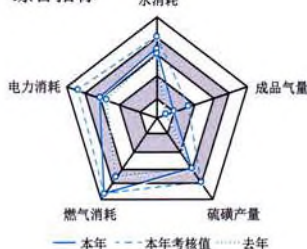
单位能耗: 2.1kg标煤/m³ 同比-0.1%

水耗趋势

监视月考核值



综合指标



本期概述: 天然气净化生产新鲜水单耗、燃料气单耗、蒸汽单耗达标, 注意水单耗临近年度考核指标, 电单耗超标。

图 13-18 报表形式输出

系统的输出设计主要关注如何向用户传达数据和信息, 所以在针对系统输出设计方面要遵循下面 7 条原则。

1. 简洁性原则

简洁性原则是指在设计系统输出时, 应该尽量避免过多的冗余信息和复杂的设计, 保持简洁明了的风格。简洁的设计可以提高用户的使用体验, 减少用户的学习成本和使用成本, 同时也可以提高系统的可维护性和可扩展性。

2. 一致性原则

一致性原则是指在设计系统输出时, 应该保持一致的设计风格 and 交互方式, 使用户在使用系统时能够更加自然和流畅地进行操作。一致的设计可以提高用户的使用效率和满意度, 同时也可以减少用户的犯错率和学习成本。在系统设计中, 保持输出一致性可以方便维护。如果系统在不同的场合下产生的输出不一致, 可能会导致维护人员难以理解系统的输出, 从而增加维护的难度和成本。例如, 如果一个电子商务网站在不同的页面上展示的商品信息不一致, 可能会使用户感到困惑, 导致用户不再信任该网站并选择去竞争对手网站购买商品。

在某些行业中, 输出的一致性是有必要的, 因为这些行业有特定的标准和法规要求。例如, 在医疗保健行业, 病人的医疗记录必须保持一致性, 以确保医疗保健质量和安全。如果病人的医疗记录不一致, 可能会对病人的健康造成威胁, 也可能违反行业标准和法规要求。

3. 可用性原则

可用性原则是指在设计系统输出时, 应该注重用户的使用体验和用户的需求, 使系统能够更加方便、易用和可靠。可用性的设计可以提高用户的满意度和忠诚度, 同时也可以减少用户的犯错率和使用成本。

4. 可访问性原则

可访问性原则是指在设计系统输出时, 应该考虑到不同用户的需求和能力, 使系统能够更

加易于访问和使用。可访问性的设计可以提高系统的可用性和可达性,同时也可以促进社会的包容性和平等性。

通过保持输出的可访问性,用户可以更容易地使用系统并且可以更好地理解输出结果,从而提高用户体验。如果系统的输出不容易访问,例如缺乏必要的标记和描述,用户可能会感到困惑和不满,降低用户的满意度和信任度。

保持输出的可访问性,可以提高系统的包容性,以满足所有用户的需求。例如,如果一个电子商务网站提供多种语言的输出,那么这个网站可以吸引更多的客户,从而增加销售额。此外,保持输出的可访问性可以帮助那些需要特殊帮助的用户,如视力障碍者、听力障碍者和运动障碍者等。

5. 可维护性原则

可维护性原则是指在设计系统输出时,应该注重系统的可维护性和可扩展性,使系统能够更加易于维护和升级。可维护性的设计可以提高系统的稳定性和可靠性,同时也可以降低系统的维护成本和升级成本。

6. 安全性原则

安全性原则是指在设计系统输出时,应该注重系统的安全性和保密性。输出安全性指的是确保系统产生的输出不会被未经授权的人员或系统访问或篡改,从而保护系统和用户的安全,使系统能够更加安全可靠地运行。安全的设计可以保护用户的隐私和数据安全,同时也可以防止系统被攻击和破坏。

在某些行业中,保持输出安全性是必要的,因为这些行业有特定的法规要求。例如,在医疗保健行业,病人的医疗记录必须保持机密性和安全性,以确保病人的隐私和医疗安全。如果未经授权的人员或系统可以访问或篡改医疗记录,可能会违反行业法规,导致罚款或其他法律后果。

7. 可扩展性原则

可扩展性原则是指在设计系统输出时,应该注重系统的可扩展性和灵活性,使系统能够更加易于扩展和适应不同的需求。可扩展的设计可以提高系统的适应性和灵活性,同时也可以降低系统的开发成本和维护成本。

13.6.3 良好的输入/输出的重要性

在软件系统的输出界面中,大多数最终效果的呈现就是界面效果。尽管黑白色是经典的颜色,但如果在设计中没有考虑其他颜色,很容易带给用户视觉上的疲劳,所以一般情况下会适当增加色彩和图像,基于用户视觉上的感染力,从而为他们减少一定的疲惫感。由于图形具有直观、形象的特点,所以能够带给用户更加直接的体验,在人机交互界面的设计中,也能够增强软件系统的易理解性。当然,在实际设计中,因为不同用户对图形有感官上的偏好,所以并不是颜色越多越好,这就要求设计人员要具备一定的艺术细胞,能够合理利用这些元素,实现动静结合的效果。除此之外,在界面效果设计中色彩的利用非常重要,如果运用得当可以给人留下深刻的印象,所以也需要最大程度发挥色彩的视觉组合,从而创造出理想的界面效果。这就要求设计人员在设计时要提前分析色彩的属性,从纯度、色相和明度去把握色彩的搭配,这

样才会带给用户不同的感觉，进一步提高软件界面的感染力，从而达到较为理想的界面效果。

因此，在系统设计过程中，输入/输出设计是至关重要的。合理的输入/输出设计可以提高系统的性能，提高用户体验，满足合规要求，保障系统的安全性和数据一致性，提高系统的可维护性。

输入/输出设计直接影响系统的性能，如果输入/输出设计不合理，可能会导致系统响应时间过长、数据处理速度慢、系统可用性降低等问题。如果系统的输入/输出不易使用，用户可能会感到困惑、不满意，从而降低用户满意度和信任度。并且，系统的输入/输出设计必须符合行业、政府机构、法规和标准的要求。例如，根据美国障碍者法案（ADA），所有网站都必须为有障碍者提供可访问性。

13.7 人机交互设计

系统的交互体验感是指系统在用户的使用、交流过程中强调互动、交互特性。交互体验贯穿用户浏览、点击等操作的各个过程，是至关重要的一环。系统人机交互设计是指设计人员将人类使用系统的方式、需求、认知特点等因素考虑在内，开发出易于理解、易于使用、易于操作的用户界面和交互方式。在系统设计中，人机交互设计是一个非常重要的环节。良好的人机交互设计可以提高用户满意度，减少培训和支持成本，提高系统的可用性和易用性，提高系统的生产力和效率，提高系统的竞争力。

13.7.1 设计目标

系统人机交互设计的目标是提高用户体验和效率，使用户能够轻松地使用系统完成任务。具体来说，系统人机交互设计的目标包括：

（1）切实满足用户的需求，功能有用、内容齐全，对用户有帮助。在任何场景中，都要先确保产品的有用性，这是用户体验的基础。

（2）产品容易上手和使用，用户用起来易操作、易理解、易学习。易用性即“用户使用成本”，用户使用任何产品都是需要学习、精力、时间等成本的，易用性就是要降低“用户成本”。

（3）降低用户的错误率和学习成本，使用户能够更容易地掌握系统的使用方法。例如像市场上的商业BI类软件很多系统需要非常专业的知识水平才能使用，无法通过改善优化人机交互的设计从而降低系统的使用门槛。

13.7.2 用户体验五层模型

良好的人际交互设计必须基于对用户需求和行为的深刻理解。在系统设计之前，设计人员需要进行调研，了解用户的需求、期望、行为和偏好等信息。在设计人员与用户之间进行交互时，需要时刻将用户放在第一位，考虑他们的需求和要求，并确保设计的系统能够满足这些需求和要求。在理解用户行为方面，设计人员需要了解用户的使用习惯和行为模式，并确保系统的设计与用户的期望一致。

“用户体验”这个词最早被广泛认知是在20世纪90年代中期，它由设计师唐纳德·诺曼

(Donald Norman) 提出。

用户体验是用户在接触和使用产品过程中所建立的感受,感受是主观的,因此用户的体验也是主观的。用户体验的五层模型由杰西·詹姆斯·加勒特(Jesse James Garrett)在《用户体验要素》里提出。他认为用户体验包含五个要素,可以用五层模型来体现,包括表现层、框架层、结构层、范围层和战略层,如图13-19所示。



图 13-19 用户体验的五层模型

1. 战略层

战略层(Strategy)主要包括用户需求和产品目标。好的产品首先要有自己的战略定位,由于个人属性的千差万别,要想用户体验良好,战略定位一定要基于某类固定需求的目标用户群体展开,任何产品都不能覆盖到用户的方方面面。

用户想要获得的就是“用户需求”,是来自外部的需求。为了确定用户需求,首先要确定产品的目标用户(Target User)。确定产品的目标用户有以下几个步骤:

- (1) 用户细分:将用户分成较小的有共同需求的组(人口统计学、消费心态档案等)。
- (2) 用户研究:知道用户是谁(问卷调查、访谈、实地考察、焦点小组、卡片分类等)。
- (3) 人物角色:从用户研究中提取出可以成为样例的虚拟人物。

通过使用以上几个步骤,能确定出用户的需求,确定出产品在战略层的内容。

例如,电商用户需要在网络上购买到物美价廉的产品,所以电商系统的整体产品目标在于构建一个C2C电商平台。

2. 范围层

当把用户需求和网站目标转变成网站应该提供给用户什么样的内容和功能时,战略就变成了范围。根据范围层(Scope)的具体内容,可以将产品大致分为工具型和内容型,有的产品兼具工具型和内容型的特点。

例如,在电商平台的功能中,需要与用户交互的功能包括搜索商品、查看详情、加入购物车、结算、支付等。内容需求则包括海量的可供挑选的商品及商品的详细介绍,如何给用户推

荐用户喜欢的商品等。

3. 结构层

在收集完用户需求并将其排列好优先级后，需要将这些分散的片段组成一个整体，这就是结构层（Structure），即创建产品功能和内容之间的关系。

结构层分为交互设计和信息架构两大部分。

交互设计是描述“可能的用户行为”，定义“系统如何配合与响应”这些用户行为。不要让用户去适应产品，而要让产品与用户互相适应，预测对方的下一步。

信息架构是关注如何将信息表达给用户，着重于设计组织分类和导航结构，让用户容易找到需要的内容。

交互设计应该至少包括概念模型和错误处理。

信息架构有两种分类体系：从上到下和从下到上。其结构可以是层级结构、矩阵结构、自然结构和线性结构。一般来说，网站的信息架构多是以上几种结构的综合，以一种结构为主，其余结构为辅。

例如，电商平台把搜索商品、查看详情、加入购物车、结算、支付等功能点组织在一起，一步一步地引导用户买到想要的商品。平台中有海量的商品，通过商品分类、搜索、扫码、系统推荐等方式可以帮助用户找到心仪的商品。通过大量完善和解决与用户的交互问题最终帮助用户找到并购买心仪的商品。

4. 框架层

框架层（Skeleton）决定某个板块或按钮等交互元素应该放在页面的什么地方。结构层中形成了大量的需求，而在框架层中，我们要更进一步地提炼这些需求，确定详细的界面外观、导航和信息设计，使晦涩的结构变得清晰明确。

在设计框架层的内容时要遵循两条原则：遵循用户日常使用习惯和恰当使用生活中的比喻。框架层设计包含界面设计、导航设计和信息设计三个方面。

（1）界面设计为用户提供做某些事的能力。做界面设计时，要选择合适的元素，让用户一眼就能看到最重要的东西。

（2）导航设计给用户去某个地方的功能。做导航设计时，要提供网页之间的跳转方法，传达元素与内容之间的关系，传达内容与页面之间的关系。

（3）信息设计是将想法传达给用户。信息设计是将各种设计元素聚合到一起，来反映用户的思路，支持他们完成任务和目标，包括视觉、分组等各个方面。

例如，在界面的什么位置放置用户常用的按钮、搜索栏等内容，底部放置导航栏，以符合用户在与系统进行交互时的行为习惯。

5. 表现层

表现层（Surface）是用户所能看见的一切，包括字体的大小、导航栏的颜色、整体给人的感觉等。在这一层，内容、功能和美学汇集在一起产生一个最终设计，从而满足其他层面的所有要求。

设计易用的界面是进行人机交互设计的核心之一。易用的界面应该简单、直观、一目了然，让用户在不经过多次试验的情况下轻松找到他们所需的功能。在设计界面时，设计人员应该考虑以下几点：

- (1) 界面元素的排布应该有逻辑性，按照用户的思维方式排布。例如，重要的功能应该放在显眼的位置。
- (2) 界面的字体、颜色等要合理搭配，不应该过于花哨或刺眼。
- (3) 界面应该尽可能简洁，只显示必要的信息，避免用户在繁杂的界面中迷失。
- (4) 界面元素应该易于识别和操作，例如，按钮的形状和颜色应该具有可识别性。

13.7.3 如何设计良好的人机交互

1. 了解用户

在设计系统时，首先要进行用户研究和需求分析。这包括以下几个关键步骤：

- (1) 用户群体划分：明确系统的目标用户群体，了解他们的特点、背景、技能水平以及使用场景等信息。
- (2) 用户需求调研：通过调查问卷、用户访谈、焦点小组等方式，收集用户需求和期望，了解他们对系统功能、界面和交互的需求。
- (3) 使用场景分析：分析用户在不同使用场景下的行为模式、任务流程和需求差异，为后续的人机交互设计提供依据。
- (4) 竞品分析：研究类似系统的竞争对手，了解他们的优点和不足，为设计提供借鉴和改进的方向。

通过以上步骤，可以深入了解用户的需求和使用习惯，为后续的人机交互设计提供指导和依据。

2. 设计交互界面

设计易用的界面是进行人机交互设计的核心之一。易用的界面应该简单、直观、一目了然，让用户在不经过多次试验的情况下轻松找到他们所需的功能。

设计符合用户习惯的交互方式可以让用户更自然、轻松地使用系统。在设计交互方式时，应该考虑用户的习惯和喜好，并尽可能地遵循用户的思维方式。例如，现在的手机应用都采用了滑动、轻扫等手势操作，这些操作都符合用户的自然习惯。设计人员应该从以下几方面考虑：

- (1) 采用简单的操作方式，避免用户需要学习新的技能或操作方式。
- (2) 采用自然的操作方式，例如，在设计桌面应用程序时，可以采用类似 Windows 操作系统的图标、文件夹等元素，这样可以让用户感到熟悉，更容易上手。
- (3) 采用统一的操作方式，避免使用不同的操作方式让用户感到混乱。

3. 提供即时反馈机制

提供即时反馈可以让用户更加容易地理解和掌握系统。在用户操作系统时，系统应该提供相应的反馈，让用户清楚地知道他们的操作是否有效，是否正在进行中或已完成。例如，在系

统下载文件时，应该提供一个下载进度条，让用户了解下载进度。

在设计反馈信息时，反馈信息应该准确、简洁，不应过于冗长，应通过明显的颜色、形状、动画等特征，让用户容易辨认。反馈信息应与用户的操作具有相关性，例如，当用户删除一张照片时，应该提供一个弹窗，提示用户确认是否删除。反馈时间应该及时，避免用户需要进行额外的等待。另外需要注意，反馈机制应该适用于不同类型的用户，例如，对于视觉障碍者，应该提供语音提示。

4. 辅助功能

前面提到应当为不同类型的用户设计不同的反馈机制，对于那些有视觉、听觉、肢体等方面障碍的人，应提供一些额外的功能。在进行人机交互设计时，应该考虑这些用户的需求，提供相应的辅助功能。例如，为了帮助视觉障碍者，可以提供语音提示、语音识别等功能。在设计辅助功能时，应该考虑辅助功能要易于使用，避免用户需要进行烦琐的设置；应该与系统的其他功能有相同的用户体验，避免用户需要进行额外的学习；辅助功能应该适用于多种障碍类型，例如，既适用于视觉障碍者，也适用于听觉障碍者。

5. 用户测试

进行用户测试是验证人机交互设计是否成功的重要手段。在设计系统时，应该进行多次用户测试，收集用户反馈，并根据反馈改进设计。参与测试的用户应该具有代表性，包括不同年龄、不同性别、不同文化背景的用户。测试应该涵盖不同的使用场景，例如，在设计手机应用时，应该测试用户在不同的场景下使用应用。反馈应该详细、具体，以便于进行设计改进。

6. 迭代优化设计

在进行人机交互设计时，不可能一步到位，需要不断迭代优化设计。进行用户测试后，会收集到更详细、具体的改进方案。通过不断迭代优化，可以不断提高系统的易用性和用户体验，满足用户需求，提高用户满意度。例如国内云计算平台在开发过程中，用户的反馈起到了非常重要的作用，通过用户的评价反复完善自己的产品，是系统设计中必不可少的一环。

7. 多语言支持

随着全球化的发展，多语言支持已经成为系统设计的必要考虑因素之一。在设计系统时，需要考虑到不同国家和地区的语言和文化差异，并提供相应的语言支持。

为不同语言提供翻译和本地化支持，可以让系统更加易用和符合用户需求，同时也可以扩大系统的用户群体。

8. 与实际应用场景相结合

在进行人机交互设计时，需要考虑实际应用场景。不同的应用场景需要设计不同的交互方式和界面布局，以便用户在特定场景下使用系统。例如，在移动设备上使用的应用程序需要设计相应的手势操作和界面布局，以适应小屏幕和移动操作的特点。

在与实际应用场景相结合时，还需要考虑用户的需求和目标。例如，在商业领域中，系统

应该设计简单、快速、直观的操作方式,以提高用户的工作效率和客户体验。

在使用系统时,用户可能会遇到各种问题,需要提供相应的帮助和文档支持。在设计系统时,需要提供清晰、易懂的帮助文档和操作说明,以使用户更好地使用系统。

帮助和文档支持包括在线帮助、操作手册、FAQ、视频教程等。通过提供丰富、易用的帮助和文档支持,可以帮助用户更好地理解系统和解决问题,提高用户体验。

9. 适当地使用动画效果

动画效果可以为用户提供更加生动、直观的交互体验。在设计系统时,可以适当使用动画效果来增加用户的参与感和体验感。例如,在转场动画中使用渐变效果可以让用户更加自然地过渡到下一个场景,在点击按钮时应用波纹效果可以让用户更加明确地感知到操作结果。

然而,过多的动画效果也会影响用户的体验。因此,在使用动画效果时需要适度,不要过度渲染,同时要确保动画效果的流畅性和自然性。

10. 适当地使用新理念和新技术

人机交互设计是一个不断发展和进步的领域,需要与时俱进,参考先进的设计理念和技术。例如,近年来,人工智能、虚拟现实、增强现实等技术逐渐应用到人机交互设计中,提供了更加高级的交互方式和用户体验。

参考先进的设计理念和技术可以不断拓展设计思路,提高设计水平,同时也可以提高系统的功能和性能。

例如:人机交互和人工智能是两个重要的领域,它们的结合可以创造出更为先进、智能的系统和应用。

首先,人工智能技术可以在人机交互中发挥重要作用。通过机器学习、深度学习等技术,可以使系统更好地理解用户的行为和需求,进而为用户提供更为个性化和精准的服务。例如,通过分析用户的历史数据,系统可以根据用户的兴趣、偏好等因素推荐符合用户需求的内容或服务,从而提高用户的满意度。另外,人工智能技术还可以帮助系统更好地理解 and 处理自然语言。自然语言处理技术可以识别、理解和生成人类语言,从而实现更为自然、智能的交互方式。例如,在智能客服中,人工智能技术可以通过自然语言处理识别用户的问题,并自动回答用户的疑问,提高客服效率。人机交互和人工智能的结合还可以在许多其他领域发挥作用,如智能家居、智能医疗、智能城市等。通过人工智能技术的加入,可以让这些系统更加智能化和人性化,为人类带来更加便利和高效的生活体验。

但是,人机交互和人工智能的结合也面临一些挑战。首先是算法的可解释性问题。人工智能模型的黑盒性质使得难以解释模型的决策过程,这会影响人机交互的可靠性和可信度。其次是数据隐私和安全问题。人工智能模型需要大量的数据支持,但数据的收集和存储往往涉及用户隐私和安全问题,需要采取相应的保护措施。

以上提到的人机交互设计原则和方法不是固定不变的,而是根据不同系统的需求和特点而有所不同。因此,在设计系统时,需要结合具体情况灵活运用和调整,以确保最终的设计结果能够达到用户的期望和需求。

此外,在进行人机交互设计时,还需要注重用户隐私和数据安全。例如,在设计登录界

面时，需要确保用户的账号和密码能够得到保护，防止用户信息泄露；在设计数据展示界面时，需要考虑数据隐私和敏感性，避免未经授权的人员获取到数据。在系统设计中注重用户隐私和数据安全，不仅可以提高用户的信任度和满意度，还可以保护用户的权益和系统的安全。

总之，人机交互设计是设计系统的重要组成部分，通过运用适当的交互设计原则和方法，可以设计出符合用户需求和习惯的易用、高效、安全、可靠的系统，提高用户体验和满意度，提高系统的竞争力和商业价值。